
La Fábrica Del Software.

ALERT !

Herramienta para clasificación de noticias

Formulación del Proyecto

Nombre del proyecto

Alert! – Plataforma Inteligente para el Monitoreo, Clasificación y Georreferenciación de Noticias mediante Procesamiento Batch

Descripción del proyecto

Alert! es una plataforma de inteligencia de información diseñada para monitorear automáticamente noticias provenientes de múltiples fuentes RSS y servicios de información digital. El sistema ejecuta procesos por lotes (Batch Processing) que recopilan noticias, extraen su contenido, identifican ubicaciones geográficas, clasifican los eventos según su nivel de severidad y generan un archivo GeoJSON estructurado.

El archivo GeoJSON constituye el documento principal para la persistencia de la información geoespacial y sirve como fuente de datos para bases de datos geográficas, sistemas de información geográfica (SIG), paneles estadísticos y aplicaciones de monitoreo territorial.

Posteriormente, la información almacenada permite generar mapas interactivos, análisis históricos, estadísticas, tendencias regionales y mecanismos de apoyo para la toma de decisiones.

Esta descripción es consistente con el propósito y los objetivos definidos en el documento de requisitos funcionales, donde se establece que Alert! recopila noticias de múltiples fuentes, las clasifica por gravedad y facilita la consulta histórica y la generación de estadísticas.

Planteamiento del problema

Las organizaciones encargadas del análisis de riesgos, la seguridad y la gestión de emergencias reciben diariamente grandes volúmenes de información provenientes de medios digitales, agencias de noticias y canales RSS.

Gran parte de esta información se encuentra distribuida en múltiples fuentes y formatos, dificultando su consolidación, análisis y georreferenciación.

Los procesos manuales para recopilar noticias presentan limitaciones como:

- Alto tiempo de procesamiento.
- Duplicidad de información.
- Dificultad para localizar eventos geográficamente.
- Baja capacidad para detectar tendencias.

- Ausencia de una base de datos geoespacial consolidada.

Como consecuencia, la información disponible no puede utilizarse de forma eficiente para el análisis territorial ni para apoyar la toma de decisiones basada en evidencia.

Justificación

El desarrollo de Alert! permitirá automatizar el proceso de recopilación y análisis de noticias mediante procesamiento por lotes.

Cada ejecución del proceso Batch realizará automáticamente:

1. Descarga de noticias desde múltiples fuentes RSS.
2. Limpieza y normalización del contenido.
3. Extracción de entidades geográficas.
4. Clasificación automática de severidad.
5. Clasificación por categorías.
6. Generación de un archivo GeoJSON.
7. Persistencia del GeoJSON como documento de referencia para una base de datos geográfica.
8. Actualización de estadísticas y mapas de incidentes.

Este enfoque reduce el tiempo de procesamiento, mejora la calidad de los datos y facilita la construcción de un histórico de eventos georreferenciados.

Objetivo general

Diseñar e implementar una plataforma inteligente que monitoree noticias mediante procesos Batch, clasifique automáticamente los eventos según su nivel de severidad, identifique su ubicación geográfica y genere archivos GeoJSON que constituyan la base documental para una base de datos geoespacial destinada al análisis histórico, estadístico y cartográfico de incidentes.

Objetivos específicos

- Implementar un sistema de captura automática de noticias desde fuentes RSS y APIs.
- Ejecutar procesos Batch para el procesamiento periódico de noticias.
- Extraer automáticamente información geográfica mediante técnicas de procesamiento de lenguaje natural.
- Clasificar cada noticia según los niveles de severidad definidos por el sistema.
- Generar archivos GeoJSON con la información georreferenciada.

- Utilizar el GeoJSON como documento persistente para alimentar una base de datos geográfica.
 - Visualizar los incidentes sobre mapas interactivos.
 - Generar estadísticas históricas y tendencias regionales.
 - Facilitar la consulta de información mediante filtros y búsquedas inteligentes.
-

Alcance

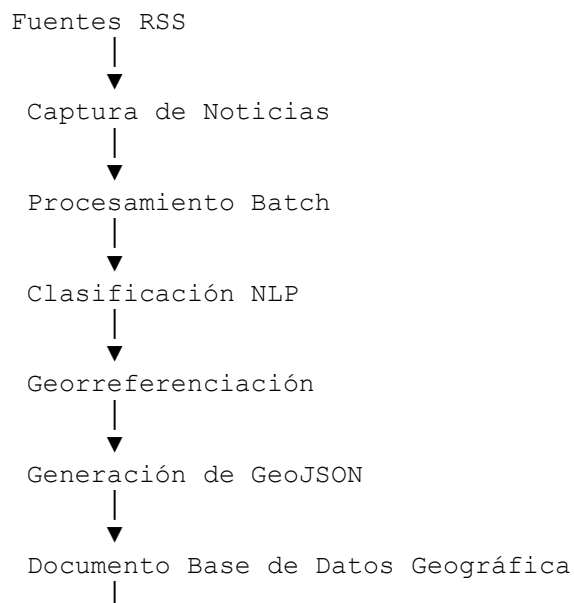
El sistema permitirá:

- Monitorear automáticamente múltiples fuentes RSS.
- Procesar noticias mediante ejecución Batch programada.
- Detectar ubicaciones geográficas.
- Clasificar noticias según ocho niveles de severidad.
- Generar archivos GeoJSON.
- Almacenar el histórico de eventos.
- Mostrar mapas interactivos.
- Consultar información histórica.
- Generar indicadores estadísticos.
- Exportar reportes.

Estas funcionalidades se corresponden con los requisitos de captura de noticias, clasificación automática, mapa de incidentes, panel de tendencias, historial de eventos y API pública descritos en la especificación funcional.

Resultado esperado

Cada ciclo Batch producirá un archivo con una estructura similar a la siguiente:



—	Dashboard
—	Mapas
—	Estadísticas
—	Reportes
—	API Pública

Esta formulación refleja el funcionamiento descrito en tus documentos y enfatiza el papel del **GeoJSON** como el principal producto del procesamiento por lotes y como base para el almacenamiento y análisis geoespacial de la información. Además, es adecuada para incluirla en un proyecto de grado del SENA o una propuesta de desarrollo de software.

1. Portada
2. Tabla de contenido
3. Introducción
4. Planteamiento del problema
5. Justificación
6. Objetivos
7. Alcance
8. Marco teórico
9. Metodología
10. Análisis de requerimientos
11. Diseño del sistema
12. Diseño de la base de datos
13. Arquitectura
14. Cronograma
15. Presupuesto
16. Riesgos
17. Resultados esperados
18. Conclusiones
19. Bibliografía
20. Anexos

Introducción

El crecimiento exponencial de la información publicada diariamente en medios digitales, portales web y canales RSS ha generado la necesidad de desarrollar herramientas que permitan recopilar, procesar y analizar grandes volúmenes de noticias de manera automática. En ámbitos como la seguridad ciudadana, la gestión del riesgo, la atención de emergencias y el análisis territorial, la disponibilidad de información oportuna, estructurada y georreferenciada constituye un elemento fundamental para apoyar la toma de decisiones.

En este contexto surge **Alert!**, una plataforma inteligente diseñada para monitorear noticias provenientes de múltiples fuentes de información mediante procesos automatizados de tipo **Batch (procesamiento por lotes)**. El sistema recopila noticias, analiza su contenido, identifica ubicaciones geográficas, clasifica los eventos según su nivel de severidad y genera un archivo en formato **GeoJSON**, el cual se convierte en el documento base para la conformación de una base de datos geoespacial. Esta información permite representar los incidentes en mapas interactivos, generar estadísticas históricas, identificar tendencias y facilitar el análisis de eventos de interés.

De acuerdo con los requisitos funcionales del proyecto, la plataforma tiene como propósito centralizar noticias provenientes de múltiples fuentes, clasificarlas automáticamente según su gravedad, mostrar alertas priorizadas, facilitar la consulta histórica de incidentes y generar estadísticas que apoyen la toma de decisiones. Asimismo, el sistema contempla la captura de noticias desde fuentes RSS y APIs, la identificación de la ubicación geográfica de los eventos, la visualización de incidentes georreferenciados y la generación de reportes e indicadores para diferentes tipos de usuarios.

La formulación de este proyecto establece los fundamentos técnicos, metodológicos y funcionales para el desarrollo de la plataforma Alert!, definiendo el problema que se pretende resolver, la justificación del proyecto, sus objetivos, alcance, requerimientos, arquitectura, metodología de desarrollo y criterios de evaluación. Además, presenta la planificación de los recursos necesarios para construir una solución tecnológica escalable, capaz de transformar información dispersa en conocimiento útil para el análisis de incidentes y la generación de inteligencia geoespacial.

Con la implementación de Alert! se busca optimizar el procesamiento masivo de noticias, reducir los tiempos de análisis, mejorar la calidad de la información georreferenciada y disponer de una base de datos histórica que permita identificar patrones espaciales y temporales de los eventos registrados. De esta manera, el proyecto

contribuye al fortalecimiento de los procesos de monitoreo, análisis e interpretación de información de seguridad y riesgo, proporcionando una herramienta tecnológica orientada al apoyo de la toma de decisiones basada en datos.

Esta introducción es adecuada para un documento de formulación de proyecto de grado del **SENA** o de una universidad y sirve como apertura para los capítulos de planteamiento del problema, justificación, objetivos y metodología.

2. Planteamiento del Problema

En la actualidad, el crecimiento continuo de los medios digitales y de las plataformas de información ha incrementado significativamente la cantidad de noticias publicadas sobre hechos relacionados con la seguridad, el orden público, los desastres naturales, las emergencias sanitarias y otros eventos de interés. Esta información se encuentra distribuida en múltiples portales web, agencias de noticias y canales RSS, lo que dificulta su recopilación, análisis y aprovechamiento de manera oportuna.

Las organizaciones encargadas del monitoreo de riesgos, la gestión de emergencias, la investigación, el análisis territorial y la seguridad requieren información confiable, actualizada y georreferenciada para comprender la evolución de los acontecimientos y apoyar los procesos de toma de decisiones. Sin embargo, en muchos casos la recopilación de noticias continúa realizándose de forma manual, consultando múltiples fuentes independientes, lo que implica un alto consumo de tiempo, duplicidad de información y una limitada capacidad para identificar patrones espaciales y temporales de los incidentes.

Adicionalmente, la mayoría de los sistemas disponibles se enfocan únicamente en mostrar noticias de manera cronológica, sin realizar un proceso automático de clasificación por niveles de severidad ni una adecuada identificación de la ubicación geográfica de los eventos. Esta situación dificulta priorizar incidentes críticos, realizar análisis históricos y generar indicadores que permitan comprender el comportamiento de los fenómenos registrados.

Otro aspecto importante es la ausencia de una estructura de datos geoespacial unificada que permita consolidar la información proveniente de diferentes fuentes. Aunque existen servicios que publican noticias o alertas, generalmente no generan un documento geográfico estandarizado que pueda incorporarse directamente a una base de datos espacial para su posterior análisis mediante Sistemas de Información Geográfica (SIG).

Frente a esta problemática surge la necesidad de desarrollar una plataforma que automatice el monitoreo de noticias mediante procesos Batch, permitiendo recopilar información desde múltiples fuentes RSS y otros servicios de información, analizar su contenido, identificar automáticamente las ubicaciones geográficas, clasificar cada evento según su nivel de severidad y generar un archivo en formato GeoJSON que sirva como documento base para la construcción de una base de datos geoespacial.

La plataforma **Alert!** busca atender esta necesidad mediante la implementación de un proceso automatizado de captura, análisis y clasificación de noticias que permita transformar grandes volúmenes de información no estructurada en datos organizados, georreferenciados y disponibles para su consulta, visualización cartográfica, análisis estadístico y generación de reportes. De acuerdo con los requisitos funcionales del proyecto, el sistema contempla la captura de noticias desde fuentes RSS y APIs, la clasificación automática por gravedad, la identificación de la ubicación geográfica, la visualización de incidentes en mapas, la generación de estadísticas y el almacenamiento del historial de eventos para su consulta posterior.

En consecuencia, el problema que aborda este proyecto puede resumirse en la siguiente pregunta de investigación:

¿Cómo desarrollar una plataforma inteligente que permita monitorear automáticamente noticias provenientes de múltiples fuentes de información mediante procesamiento por lotes (Batch), clasificarlas según su nivel de severidad, georreferenciarlas y generar archivos GeoJSON que sirvan como base para una base de datos geoespacial destinada al análisis histórico, estadístico y cartográfico de incidentes?

3. Justificación

El crecimiento constante de la información publicada en medios digitales y fuentes RSS ha incrementado la necesidad de contar con herramientas capaces de recopilar, procesar y analizar grandes volúmenes de datos de manera automática. En el ámbito de la seguridad, la gestión del riesgo, la investigación y el análisis territorial, disponer de información organizada y georreferenciada es fundamental para identificar eventos relevantes, establecer prioridades y apoyar la toma de decisiones.

Actualmente, gran parte de la información sobre incidentes de seguridad, conflictos, desastres naturales, emergencias sanitarias y otros acontecimientos de interés se encuentra distribuida en múltiples portales de noticias y servicios de información. La consulta manual de estas fuentes demanda un esfuerzo considerable, dificulta la consolidación de los datos y limita la posibilidad de realizar análisis oportunos e integrales.

El proyecto **Alert!** se plantea como una solución tecnológica que automatiza el monitoreo de noticias mediante procesos Batch (procesamiento por lotes), permitiendo recopilar información desde diversas fuentes, analizar el contenido de las noticias, identificar automáticamente las ubicaciones geográficas, clasificar los eventos de acuerdo con una escala de severidad y generar un archivo en formato **GeoJSON**. Este archivo constituye un documento geoespacial estructurado que puede utilizarse como base para una base de datos geográfica, facilitando la consulta, el análisis histórico y la representación cartográfica de los incidentes.

Desde el punto de vista técnico, la plataforma permitirá reducir significativamente el tiempo requerido para recopilar y procesar información proveniente de múltiples fuentes, disminuyendo la intervención manual y mejorando la calidad, consistencia y organización de los datos. Asimismo, la generación automática de archivos GeoJSON

favorecerá la interoperabilidad con Sistemas de Información Geográfica (SIG), plataformas cartográficas y herramientas de análisis espacial.

En el ámbito funcional, el proyecto incorpora procesos para la captura automática de noticias desde fuentes RSS y APIs, la clasificación por niveles de severidad, la identificación de ubicaciones geográficas, la visualización de incidentes sobre mapas, la generación de estadísticas, el almacenamiento del historial de eventos y la producción de reportes para diferentes tipos de usuarios, funcionalidades definidas dentro de los requisitos funcionales del sistema.

Desde una perspectiva social, Alert! puede convertirse en una herramienta de apoyo para entidades públicas, organismos de gestión del riesgo, centros de investigación, observatorios, medios de comunicación y organizaciones interesadas en el seguimiento de incidentes y la construcción de análisis territoriales basados en información pública. Al consolidar y estructurar la información proveniente de diferentes fuentes, la plataforma facilita la identificación de tendencias, el análisis espacial de los eventos y la elaboración de estudios que contribuyan a una mejor comprensión de los fenómenos registrados.

En el ámbito académico, el desarrollo de este proyecto permitirá aplicar conocimientos adquiridos durante la formación en Análisis y Desarrollo de Software, integrando disciplinas como ingeniería de software, desarrollo web, procesamiento de datos, bases de datos, sistemas de información geográfica, procesamiento de lenguaje natural y visualización de información. Además, representa una oportunidad para implementar buenas prácticas de diseño, desarrollo y documentación de software, fortaleciendo las competencias profesionales del equipo desarrollador.

Finalmente, la implementación de Alert! contribuirá a disponer de una plataforma escalable y adaptable que transforme grandes volúmenes de información no estructurada en datos georreferenciados de valor para el análisis y la generación de conocimiento. De esta manera, el proyecto aporta una solución innovadora para el monitoreo automatizado de noticias, apoyando procesos de análisis, consulta histórica y toma de decisiones mediante el uso de información organizada, clasificada y geográficamente referenciada.

4. Objetivo General

Diseñar e implementar una plataforma inteligente denominada **Alert!** que permita monitorear automáticamente noticias provenientes de múltiples fuentes de información mediante procesos Batch (procesamiento por lotes), con el fin de recopilar, analizar, clasificar y georreferenciar eventos de interés, generando archivos en formato **GeoJSON** como documento base para una base de datos geoespacial que facilite la visualización cartográfica, el análisis histórico, la generación de estadísticas y el apoyo a la toma de decisiones.

5. Objetivos Específicos

1. Analizar los requerimientos funcionales y no funcionales del sistema Alert! para definir las necesidades del proceso de monitoreo, clasificación y georreferenciación de noticias provenientes de múltiples fuentes de información.
2. Diseñar la arquitectura de software y el modelo de datos que soporten la captura, procesamiento, almacenamiento y consulta de noticias clasificadas y georreferenciadas.
3. Desarrollar un módulo de captura automática de noticias que obtenga información desde fuentes RSS y otras APIs, consolidando los datos en un formato estructurado para su procesamiento.
4. Implementar un proceso Batch (procesamiento por lotes) que analice periódicamente las noticias recopiladas, normalice la información y prepare los datos para su clasificación y georreferenciación.
5. Desarrollar un mecanismo de clasificación automática que determine el nivel de severidad de cada noticia y la categorice de acuerdo con el tipo de incidente identificado.
6. Implementar un módulo de identificación de ubicaciones geográficas que permita asociar cada noticia con coordenadas espaciales para su representación cartográfica.
7. Generar archivos en formato GeoJSON como documento base para la conformación de una base de datos geoespacial, garantizando la integridad y consistencia de la información procesada.
8. Desarrollar una interfaz web que permita visualizar las noticias clasificadas mediante mapas interactivos, filtros de búsqueda, indicadores estadísticos y consultas históricas.
9. Implementar funcionalidades para la generación de reportes y estadísticas que faciliten el análisis temporal y espacial de los eventos registrados.
10. Realizar pruebas funcionales, de integración y de rendimiento para verificar el correcto funcionamiento de la plataforma, asegurando el cumplimiento de los requisitos establecidos para el proyecto Alert!.

6. Alcance del Proyecto

El proyecto **Alert!** comprende el diseño, desarrollo e implementación de una plataforma inteligente para el monitoreo automatizado de noticias provenientes de múltiples fuentes de información, mediante procesos de procesamiento por lotes (Batch), con el propósito de recopilar, analizar, clasificar y georreferenciar eventos de interés.

La solución permitirá integrar información obtenida desde canales RSS y otras fuentes de datos, procesándola automáticamente para identificar el contenido relevante, determinar la ubicación geográfica de los eventos y clasificarlos de acuerdo con una escala de severidad previamente definida. Como resultado del procesamiento, el sistema generará archivos en formato **GeoJSON**, los cuales constituirán el documento base para la conformación de una base de datos geoespacial destinada al almacenamiento, consulta y análisis de la información.

Dentro del alcance funcional del proyecto se contempla el desarrollo de los siguientes módulos:

- Captura automática de noticias desde múltiples fuentes RSS y APIs.
- Procesamiento Batch para la recopilación, limpieza, normalización y consolidación de la información.
- Clasificación automática de noticias según categorías y niveles de severidad.
- Identificación de ubicaciones geográficas mediante técnicas de procesamiento de texto.
- Generación automática de archivos GeoJSON con la información georreferenciada.
- Almacenamiento del historial de noticias procesadas para consultas posteriores.
- Visualización de incidentes sobre mapas interactivos con filtros por ubicación, categoría, fecha y nivel de severidad.
- Panel estadístico para el análisis de tendencias, distribución geográfica e indicadores históricos.
- Generación de reportes en diferentes formatos para apoyar procesos de análisis.
- Administración de usuarios, categorías, fuentes de información y configuración general del sistema.
- Exposición de una API para consultar alertas, estadísticas e incidentes históricos.

Asimismo, el proyecto contempla el diseño de una arquitectura de software escalable, el desarrollo de la base de datos, la implementación de mecanismos de autenticación y control de acceso, la realización de pruebas funcionales e integración, así como el despliegue de la plataforma en un entorno de ejecución que permita su operación y mantenimiento.

Como producto principal del sistema, cada ejecución del proceso Batch generará un archivo GeoJSON estructurado, el cual servirá como fuente documental para alimentar una base de datos geográfica y soportar procesos de análisis espacial, generación de mapas temáticos, estadísticas históricas y visualización de incidentes.

Exclusiones del alcance

Con el fin de delimitar el proyecto, no se contempla en esta versión el desarrollo de las siguientes funcionalidades:

- Integración con sistemas propietarios de entidades gubernamentales o privadas.
- Publicación automática de contenido en redes sociales.
- Predicción de eventos mediante modelos avanzados de inteligencia artificial.
- Recolección de información proveniente de redes sociales en tiempo real.
- Aplicaciones móviles nativas para dispositivos Android o iOS.
- Modificación automática del contenido original publicado por las fuentes de información.
- Reemplazo de sistemas oficiales de gestión de emergencias o seguridad.

El alcance definido permitirá disponer de una plataforma funcional para el monitoreo, clasificación y georreferenciación de noticias, proporcionando una base sólida para futuras ampliaciones, como la incorporación de nuevos modelos de análisis, fuentes

adicionales de información, algoritmos de inteligencia artificial y herramientas avanzadas de análisis geoespacial.

7. Marco Teórico

7.1 Ingeniería de Software

La ingeniería de software es la disciplina encargada de aplicar principios científicos, metodologías y buenas prácticas para el análisis, diseño, desarrollo, implementación y mantenimiento de sistemas informáticos. Su propósito es garantizar que el software satisfaga las necesidades de los usuarios mediante procesos organizados que aseguren calidad, confiabilidad, seguridad y facilidad de mantenimiento.

En el proyecto **Alert!**, la ingeniería de software proporciona el marco metodológico para definir los requerimientos del sistema, diseñar su arquitectura, implementar los módulos funcionales y realizar pruebas que aseguren el correcto funcionamiento de la plataforma.

7.2 Monitoreo Automatizado de Información

El monitoreo automatizado consiste en la recopilación continua de información proveniente de diferentes fuentes digitales mediante procesos informáticos que permiten reducir la intervención humana.

En este proyecto, el monitoreo se realiza mediante la consulta periódica de canales RSS y otras fuentes de información, permitiendo recopilar noticias relacionadas con seguridad, orden público, conflictos, desastres naturales y otros eventos de interés.

La automatización del proceso mejora la oportunidad de la información, reduce tiempos de procesamiento y facilita la construcción de un repositorio histórico de noticias.

7.3 Procesamiento por Lotes (Batch Processing)

El procesamiento por lotes o **Batch Processing** es una técnica mediante la cual un conjunto de datos se procesa automáticamente en intervalos programados, sin requerir interacción permanente del usuario.

En **Alert!**, esta metodología permite:

- Capturar noticias desde múltiples fuentes.
- Normalizar la información obtenida.
- Eliminar registros duplicados.
- Analizar el contenido.
- Clasificar la severidad.
- Identificar ubicaciones geográficas.

- Generar automáticamente archivos GeoJSON.

Este enfoque garantiza un procesamiento eficiente de grandes volúmenes de información y facilita la actualización periódica de la base documental del sistema.

7.4 Canales RSS

RSS (Really Simple Syndication) es un formato basado en XML utilizado para distribuir contenido actualizado de sitios web y medios de comunicación.

Los canales RSS permiten acceder automáticamente a nuevas publicaciones sin necesidad de consultar manualmente cada portal de noticias.

La plataforma Alert! utiliza estas fuentes como principal mecanismo para la captura de información, permitiendo integrar noticias provenientes de múltiples medios en un único proceso automatizado, tal como se contempla en los requisitos funcionales del sistema.

7.5 Procesamiento de Lenguaje Natural (NLP)

El Procesamiento de Lenguaje Natural (Natural Language Processing, NLP) es una rama de la inteligencia artificial dedicada al análisis e interpretación del lenguaje humano mediante algoritmos computacionales.

Dentro del proyecto Alert!, el NLP se emplea para:

- Analizar titulares y contenido de las noticias.
- Detectar palabras clave.
- Identificar categorías.
- Reconocer entidades geográficas.
- Determinar niveles de severidad.

Estas capacidades permiten transformar texto no estructurado en información organizada y útil para el análisis.

7.6 Clasificación Automática de Noticias

La clasificación automática consiste en asignar una categoría o nivel de prioridad a una noticia mediante reglas, algoritmos o modelos computacionales.

Según la especificación funcional del proyecto, Alert! clasifica automáticamente cada noticia utilizando una escala de severidad comprendida entre los niveles 1 y 8, permitiendo priorizar eventos críticos y facilitar su análisis.

La clasificación considera aspectos como:

- Tipo de incidente.
- Número de víctimas.
- Nivel de impacto.
- Palabras clave.
- Contexto de la noticia.
- Ubicación del evento.

7.7 Georreferenciación

La georreferenciación es el proceso mediante el cual un evento o elemento se asocia con una ubicación geográfica específica mediante coordenadas.

En Alert!, la georreferenciación constituye uno de los procesos principales del sistema, ya que permite localizar espacialmente cada incidente identificado dentro de las noticias procesadas.

La información geográfica obtenida facilita:

- Visualización en mapas.
- Consultas espaciales.
- Estadísticas regionales.
- Análisis territorial.
- Seguimiento histórico de eventos.

7.8 Formato GeoJSON

GeoJSON es un estándar abierto basado en JSON para representar información geográfica.

Permite almacenar:

- Puntos.
- Líneas.
- Polígonos.
- Colecciones de objetos geográficos.

Cada registro puede contener atributos como:

- Nombre del evento.
- Fecha.
- Categoría.
- Nivel de severidad.
- Fuente.
- Coordenadas geográficas.

En Alert!, el archivo GeoJSON representa el principal producto generado por cada proceso Batch y constituye el documento base para alimentar la base de datos geoespacial y soportar la visualización de incidentes en mapas interactivos.

7.9 Bases de Datos Geoespaciales

Las bases de datos geoespaciales permiten almacenar información geográfica junto con atributos descriptivos.

Estas bases de datos facilitan operaciones como:

- Consultas espaciales.
- Búsquedas por proximidad.
- Análisis de distribución.
- Estadísticas territoriales.
- Generación de mapas temáticos.

En el proyecto Alert!, los archivos GeoJSON constituyen la fuente documental para la construcción y actualización de una base de datos geográfica destinada al almacenamiento histórico de los incidentes procesados.

7.10 Sistemas de Información Geográfica (SIG)

Un Sistema de Información Geográfica (SIG) es un conjunto de herramientas destinadas a capturar, almacenar, analizar y representar información geográfica.

Los SIG permiten:

- Visualizar mapas interactivos.
- Analizar distribución espacial.
- Identificar zonas críticas.
- Generar indicadores territoriales.
- Apoyar procesos de planeación y toma de decisiones.

La información generada por Alert! puede integrarse fácilmente con plataformas SIG gracias al uso del formato GeoJSON.

7.11 Visualización Cartográfica

La visualización cartográfica facilita la interpretación de grandes volúmenes de información mediante mapas temáticos.

Dentro de la plataforma Alert!, los usuarios podrán visualizar:

- Ubicación de incidentes.
- Nivel de severidad.
- Categoría del evento.
- Fecha de ocurrencia.
- Fuente de información.

Los requisitos funcionales contemplan la visualización de incidentes georreferenciados y la posibilidad de consultar sus detalles mediante un mapa interactivo.

7.12 Estadística Descriptiva y Análisis Histórico

La estadística descriptiva permite resumir y representar información mediante indicadores cuantitativos y gráficos.

Alert! incorpora herramientas para generar:

- Estadísticas diarias.
- Estadísticas semanales.
- Estadísticas mensuales.
- Tendencias regionales.
- Frecuencia por categoría.
- Distribución por niveles de severidad.

Estos análisis permiten identificar comportamientos recurrentes y apoyar la toma de decisiones basada en evidencia.

7.13 Arquitectura Cliente–Servidor

La plataforma Alert! se fundamenta en una arquitectura Cliente–Servidor, en la cual el procesamiento principal se realiza en el servidor, mientras que la interfaz web permite a los usuarios consultar, filtrar y visualizar la información.

Esta arquitectura facilita:

- Centralización de datos.
 - Escalabilidad.
 - Mantenimiento del sistema.
 - Acceso remoto mediante navegador web.
 - Integración con otros servicios mediante APIs.
-

7.14 Desarrollo Web

El desarrollo web integra tecnologías orientadas a construir aplicaciones accesibles desde navegadores.

En el proyecto Alert! se emplean herramientas que permiten desarrollar módulos para:

- Gestión de usuarios.
- Consulta de noticias.
- Visualización de mapas.
- Reportes.
- Paneles estadísticos.
- Administración del sistema.

Estas tecnologías proporcionan una interfaz intuitiva y facilitan el acceso a la información procesada por la plataforma.

7.15 Relación del Marco Teórico con el Proyecto

Los conceptos expuestos constituyen la base científica y tecnológica para el desarrollo de Alert!. La combinación de ingeniería de software, procesamiento por lotes, monitoreo automatizado, procesamiento de lenguaje natural, georreferenciación, formatos geoespaciales y sistemas de información geográfica permite construir una plataforma capaz de transformar grandes volúmenes de noticias en información organizada, clasificada y georreferenciada. Como resultado, el sistema facilita la generación de archivos GeoJSON, la conformación de una base de datos geográfica, la elaboración de estadísticas, la visualización cartográfica y el análisis histórico de incidentes, proporcionando información de apoyo para la consulta y la toma de decisiones.

8. Marco Conceptual

El marco conceptual reúne los principales conceptos utilizados en el desarrollo del proyecto **Alert!**, con el propósito de establecer un lenguaje común que facilite la comprensión de los procesos, tecnologías y funcionalidades implementadas en la plataforma.

Alerta

Es la notificación generada por el sistema cuando se detecta un evento que, debido a su nivel de severidad o ubicación geográfica, requiere atención prioritaria. Las alertas permiten informar de manera oportuna sobre incidentes relevantes y apoyar la toma de decisiones.

Batch (Procesamiento por Lotes)

Es una técnica de procesamiento mediante la cual un conjunto de datos es ejecutado automáticamente en intervalos programados sin intervención directa del usuario. En el proyecto Alert!, este proceso permite recopilar noticias desde diferentes fuentes, analizarlas, clasificarlas y generar un archivo GeoJSON con la información procesada.

Canal RSS

RSS (Really Simple Syndication) es un formato de distribución de contenido basado en XML que permite publicar y consultar automáticamente las actualizaciones de sitios web y medios de comunicación. Los canales RSS constituyen una de las principales fuentes de información utilizadas por Alert! para el monitoreo automatizado de noticias.

Clasificación Automática

Proceso mediante el cual el sistema analiza el contenido de una noticia y determina su categoría y nivel de severidad utilizando reglas de procesamiento y análisis de texto, reduciendo la intervención manual y mejorando la consistencia de los resultados.

Clasificación por Severidad

Proceso mediante el cual cada noticia es evaluada según el impacto del evento reportado y se asigna un nivel de severidad. El proyecto Alert! utiliza una escala de ocho niveles, desde información general hasta eventos catastróficos, para priorizar la visualización y el análisis de los incidentes.

Dashboard

Panel de control que presenta indicadores, estadísticas y gráficos sobre las noticias procesadas por el sistema. Facilita el seguimiento de tendencias, la consulta de eventos y el análisis del comportamiento de los incidentes registrados.

Evento

Hecho o incidente reportado en una noticia que puede corresponder a situaciones relacionadas con seguridad, orden público, conflictos, emergencias, desastres naturales, pandemias u otros acontecimientos de interés para el sistema.

Fuente de Información

Medio digital, portal web, servicio RSS o API desde el cual la plataforma obtiene las noticias que serán procesadas y clasificadas.

GeoJSON

Formato estándar basado en JSON utilizado para representar información geográfica. En Alert!, cada proceso Batch genera un archivo GeoJSON que contiene la ubicación geográfica de los eventos y sus atributos asociados, convirtiéndose en el documento base para la base de datos geoespacial.

Georreferenciación

Proceso mediante el cual una noticia es asociada con una ubicación geográfica específica mediante coordenadas, permitiendo su representación en mapas y su análisis espacial.

Historial de Eventos

Repositorio donde se almacenan las noticias procesadas junto con su fecha, categoría, nivel de severidad, fuente y ubicación geográfica. Este historial facilita la consulta posterior y el análisis de tendencias a lo largo del tiempo.

Incidente

Situación o acontecimiento identificado en una noticia que puede afectar a una población, infraestructura o territorio. Los incidentes constituyen la unidad principal de análisis dentro de la plataforma Alert!.

Inteligencia Geoespacial

Proceso de transformación de datos geográficos en información útil para apoyar el análisis territorial y la toma de decisiones. En el proyecto Alert!, la inteligencia geoespacial se obtiene mediante la integración de noticias clasificadas con información cartográfica.

Mapa de Incidentes

Representación cartográfica de los eventos procesados por la plataforma. Permite visualizar la distribución espacial de los incidentes, aplicar filtros y consultar información detallada de cada registro, conforme a las funcionalidades definidas para el sistema.

Monitoreo Automatizado

Proceso continuo de recopilación de información mediante herramientas informáticas que consultan fuentes digitales sin intervención manual, garantizando una actualización periódica de los datos disponibles para el sistema.

Normalización de Datos

Proceso mediante el cual la información obtenida desde diferentes fuentes es transformada a una estructura uniforme, eliminando inconsistencias, duplicidades y diferencias de formato para facilitar su almacenamiento y análisis.

Procesamiento de Lenguaje Natural (NLP)

Conjunto de técnicas utilizadas para analizar el lenguaje humano en formato digital. En Alert!, estas técnicas permiten identificar palabras clave, categorías, entidades geográficas y otros elementos relevantes presentes en las noticias.

Repositorio Geoespacial

Conjunto organizado de datos geográficos que almacena la información generada por la plataforma. Este repositorio permite realizar consultas espaciales, generar estadísticas y construir mapas temáticos.

Sistema de Información Geográfica (SIG)

Conjunto de herramientas destinadas a capturar, almacenar, administrar, analizar y visualizar información geográfica. Los archivos GeoJSON generados por Alert! pueden ser integrados con plataformas SIG para ampliar las capacidades de análisis territorial.

Visualización Cartográfica

Proceso mediante el cual la información georreferenciada es presentada sobre mapas digitales utilizando marcadores, colores y capas temáticas que facilitan la interpretación de los datos y la identificación de patrones espaciales.

Usuario Analista

Actor del sistema encargado de revisar, validar y, cuando sea necesario, ajustar la clasificación automática de las noticias para garantizar la calidad y confiabilidad de la información procesada.

Base de Datos Geoespacial

Repositorio estructurado que almacena la información georreferenciada generada por el sistema. En el proyecto Alert!, esta base de datos se alimenta a partir de los archivos GeoJSON producidos por los procesos Batch y constituye el soporte para las consultas históricas, la generación de estadísticas y la visualización de incidentes en mapas interactivos.

9. Marco Legal

9.1 Introducción

El desarrollo e implementación de la plataforma **Alert!** debe realizarse en cumplimiento del marco normativo colombiano relacionado con el desarrollo de software, el tratamiento de datos, la seguridad de la información, los derechos de autor y el acceso a la información pública.

Dado que el sistema recopila noticias provenientes de fuentes públicas mediante canales RSS y otros servicios de información, clasifica automáticamente los eventos, genera archivos GeoJSON y almacena información georreferenciada para su análisis, resulta indispensable garantizar que el tratamiento de la información se realice respetando la legislación vigente y los principios éticos asociados al uso de tecnologías de la información.

9.2 Constitución Política de Colombia de 1991

La Constitución Política constituye el principal fundamento jurídico para el desarrollo del proyecto.

Entre los artículos más relevantes se encuentran:

- **Artículo 15:** reconoce el derecho fundamental al habeas data, la intimidad y la protección de la información personal.
- **Artículo 20:** garantiza la libertad de expresión y el derecho a recibir información veraz e imparcial.
- **Artículo 74:** establece que el acceso a los documentos públicos es un derecho de todas las personas, salvo las excepciones previstas por la ley.

Estos principios permiten utilizar información publicada por medios de comunicación y organismos oficiales, respetando los derechos fundamentales y la normativa vigente.

9.3 Ley 527 de 1999

La Ley 527 de 1999 regula el comercio electrónico, los mensajes de datos, las firmas digitales y la validez jurídica de la información electrónica.

En el proyecto Alert!, esta ley respalda el tratamiento de información digital obtenida mediante canales electrónicos y la generación de documentos digitales utilizados durante el procesamiento de noticias.

9.4 Ley 594 de 2000 (Ley General de Archivos)

Esta norma establece principios para la administración, conservación y organización de documentos.

Aunque Alert! no constituye un sistema de gestión documental institucional, el almacenamiento organizado de noticias, archivos GeoJSON e históricos de eventos debe considerar buenas prácticas de conservación y administración de la información.

9.5 Ley 603 de 2000

Esta ley promueve la protección de los derechos de autor sobre programas de computador y demás obras intelectuales.

El código fuente desarrollado para la plataforma Alert! constituye una obra protegida por la legislación colombiana sobre propiedad intelectual.

9.6 Ley 1266 de 2008

La Ley 1266 regula el derecho al habeas data y el manejo responsable de la información contenida en bases de datos.

Aunque Alert! procesa principalmente información proveniente de fuentes públicas, cualquier dato personal que eventualmente sea almacenado deberá ser tratado conforme a los principios de legalidad, finalidad, seguridad y confidencialidad establecidos en la legislación.

9.7 Ley 1273 de 2009

Esta ley creó nuevos tipos penales relacionados con la protección de la información y de los datos, estableciendo delitos informáticos como:

- Acceso abusivo a sistemas informáticos.
- Daño informático.
- Interceptación de datos.
- Uso de software malicioso.
- Violación de datos personales.

La plataforma deberá implementar mecanismos de autenticación, control de acceso y protección de la información para minimizar riesgos asociados a la seguridad informática.

9.8 Ley 1581 de 2012

La Ley 1581 establece el régimen general de protección de datos personales en Colombia.

En caso de que Alert! almacene información correspondiente a usuarios registrados, administradores o analistas, el tratamiento de dichos datos deberá cumplir los principios de:

- Legalidad.
- Finalidad.

- Libertad.
- Transparencia.
- Acceso restringido.
- Seguridad.
- Confidencialidad.

Asimismo, deberá contar con políticas de tratamiento de datos personales cuando sea aplicable.

9.9 Decreto 1377 de 2013

Este decreto reglamenta parcialmente la Ley 1581 de 2012 y define procedimientos relacionados con:

- Autorización para el tratamiento de datos.
- Derechos de los titulares.
- Políticas de privacidad.
- Responsables y encargados del tratamiento.

Estas disposiciones deberán observarse en caso de que la plataforma gestione cuentas de usuarios o almacene información personal.

9.10 Ley 1712 de 2014 (Ley de Transparencia y Acceso a la Información Pública)

Esta ley promueve el acceso a la información pública y la transparencia en las actuaciones del Estado.

Alert! utiliza información publicada por medios y fuentes abiertas, por lo que esta normativa constituye un referente importante para el uso responsable de información pública, siempre respetando las restricciones legales y los derechos de autor de las fuentes originales.

9.11 Derechos de Autor

El proyecto deberá respetar la legislación colombiana sobre propiedad intelectual.

Aunque Alert! recopila noticias provenientes de diferentes medios, la plataforma no modifica ni reclama la autoría del contenido original. La información se utiliza para fines de clasificación, análisis y visualización, manteniendo la referencia a la fuente correspondiente cuando esta sea mostrada al usuario.

9.12 Gobierno Digital

La Política de Gobierno Digital en Colombia impulsa el uso de tecnologías de la información para mejorar la eficiencia, interoperabilidad, transparencia y acceso a los servicios digitales.

El desarrollo de Alert! se alinea con estos principios al facilitar el procesamiento automatizado de información pública, la generación de datos geoespaciales y el acceso organizado a información útil para procesos de análisis y toma de decisiones.

9.13 Normas Técnicas Aplicables

Durante el desarrollo del software se recomienda adoptar buenas prácticas establecidas en estándares internacionales, entre ellos:

- **ISO/IEC 25010:** Modelo de calidad del producto software.
- **ISO/IEC 12207:** Procesos del ciclo de vida del software.
- **ISO/IEC 27001:** Sistema de Gestión de Seguridad de la Información.
- **ISO/IEC 29119:** Pruebas de software.
- **RFC 7946:** Especificación oficial del formato GeoJSON para el intercambio de información geográfica.

La aplicación de estos estándares contribuirá a mejorar la calidad, seguridad, interoperabilidad y mantenibilidad de la plataforma.

9.14 Aplicación del Marco Legal al Proyecto Alert!

El desarrollo de Alert! deberá garantizar el cumplimiento de la normativa colombiana aplicable al tratamiento de información digital, la protección de datos personales, la seguridad informática y la propiedad intelectual. Asimismo, deberá asegurar que la recopilación de noticias desde fuentes públicas se realice respetando las condiciones de uso de dichas fuentes y que la información procesada sea utilizada con fines de análisis, clasificación y georreferenciación.

El cumplimiento de este marco legal permitirá que la plataforma opere bajo principios de legalidad, transparencia, responsabilidad y protección de la información, fortaleciendo su confiabilidad y facilitando su utilización en proyectos académicos, de investigación o institucionales.

10. Estado del Arte

10.1 Introducción

El crecimiento de Internet y de los medios digitales ha incrementado significativamente la cantidad de información publicada diariamente sobre acontecimientos relacionados

con seguridad, conflictos, desastres naturales, salud pública, fenómenos ambientales y otros eventos de interés. Esta situación ha impulsado el desarrollo de plataformas especializadas en la recopilación, procesamiento y análisis automatizado de información proveniente de múltiples fuentes, permitiendo transformar grandes volúmenes de datos en conocimiento útil para la toma de decisiones.

En los últimos años, los avances en inteligencia artificial, procesamiento de lenguaje natural, minería de textos, análisis geoespacial y sistemas de información geográfica han permitido construir herramientas capaces de identificar patrones, clasificar información y representar eventos sobre mapas digitales en tiempo casi real. Estas tecnologías constituyen la base sobre la cual se desarrolla la plataforma **Alert!**.

10.2 Evolución de los Sistemas de Monitoreo de Noticias

Los primeros sistemas de monitoreo de noticias se limitaban a recopilar información desde un número reducido de fuentes y presentarla de forma cronológica para su consulta.

Con el crecimiento de Internet y la aparición de los canales RSS, fue posible automatizar la recopilación de información procedente de múltiples medios de comunicación, reduciendo la necesidad de realizar búsquedas manuales.

Posteriormente, el desarrollo de algoritmos de procesamiento de lenguaje natural permitió incorporar funciones como:

- Clasificación automática de noticias.
- Identificación de palabras clave.
- Detección de entidades.
- Reconocimiento de ubicaciones geográficas.
- Análisis temático.
- Organización automática del contenido.

Actualmente, las plataformas más avanzadas integran técnicas de inteligencia artificial, aprendizaje automático y análisis geoespacial para apoyar procesos de monitoreo y análisis de información.

10.3 Plataformas Internacionales de Monitoreo

Diversas organizaciones internacionales han desarrollado sistemas para el monitoreo y análisis de información de interés global.

Entre los más representativos se encuentran:

- Google News.
- GDELT Project.
- ReliefWeb.

- ACLED (Armed Conflict Location & Event Data).
- Global Disaster Alert and Coordination System (GDACS).

Estas plataformas recopilan información desde múltiples fuentes, clasifican eventos y presentan indicadores estadísticos y geográficos para facilitar el análisis de fenómenos de alcance regional y mundial.

Aunque ofrecen un amplio volumen de información, muchas de estas soluciones están orientadas a contextos específicos o utilizan metodologías propias para la clasificación de eventos.

10.4 Sistemas de Información Geográfica

Los Sistemas de Información Geográfica (SIG) han evolucionado hasta convertirse en herramientas fundamentales para el análisis espacial de grandes volúmenes de información.

Aplicaciones como:

- QGIS.
- ArcGIS.
- Google Maps Platform.
- Leaflet.
- OpenLayers.

permiten representar información geográfica mediante mapas interactivos, realizar consultas espaciales y generar análisis territoriales.

El uso del formato GeoJSON ha favorecido la interoperabilidad entre estas plataformas y diferentes aplicaciones de software.

10.5 Procesamiento de Lenguaje Natural

El Procesamiento de Lenguaje Natural (Natural Language Processing - NLP) ha experimentado un crecimiento significativo gracias al desarrollo de modelos computacionales capaces de comprender grandes cantidades de texto.

Actualmente estas tecnologías permiten:

- Reconocer entidades nombradas.
- Detectar ubicaciones.
- Identificar organizaciones.
- Analizar sentimientos.
- Clasificar documentos.
- Extraer información relevante.

Estas capacidades resultan fundamentales para proyectos orientados al análisis automático de noticias.

10.6 Inteligencia Artificial Aplicada al Análisis de Noticias

La inteligencia artificial ha permitido automatizar tareas que anteriormente requerían intervención humana.

En plataformas de monitoreo de noticias se emplea para:

- Clasificar eventos.
- Detectar noticias duplicadas.
- Priorizar información.
- Identificar tendencias.
- Generar estadísticas.
- Detectar anomalías.

Estas funcionalidades mejoran la velocidad y precisión del procesamiento de grandes volúmenes de información.

10.7 Formatos Geoespaciales

La representación de información geográfica ha evolucionado hacia formatos abiertos que facilitan el intercambio de datos entre diferentes plataformas.

Entre los formatos más utilizados se encuentran:

- GeoJSON.
- Shapefile.
- KML.
- GeoPackage.

El formato GeoJSON se ha convertido en uno de los estándares más utilizados en aplicaciones web debido a su simplicidad, compatibilidad con JSON y facilidad de integración con bibliotecas cartográficas.

10.8 Tendencias Tecnológicas Actuales

Las tendencias actuales en el desarrollo de plataformas de monitoreo de información incluyen:

- Automatización mediante procesamiento Batch y procesamiento en tiempo real.
- Integración con APIs abiertas.
- Inteligencia Artificial Generativa.

- Aprendizaje Automático.
- Sistemas de Información Geográfica.
- Big Data.
- Computación en la nube.
- Visualización interactiva de datos.
- Paneles de inteligencia de negocios.
- Integración mediante servicios REST.

Estas tecnologías permiten construir plataformas escalables capaces de procesar millones de registros diarios.

10.9 Aporte del Proyecto Alert!

Aunque existen diversas plataformas para el seguimiento de noticias y eventos, muchas se enfocan en la visualización de información o en la recopilación de datos, mientras que **Alert!** propone una arquitectura orientada a automatizar todo el ciclo de procesamiento de la información.

Su principal aporte consiste en integrar, dentro de una única plataforma, las siguientes funcionalidades:

- Captura automática de noticias desde múltiples fuentes RSS.
- Procesamiento Batch para la consolidación y normalización de la información.
- Clasificación automática de eventos mediante niveles de severidad.
- Identificación de ubicaciones geográficas.
- Generación automática de archivos GeoJSON.
- Construcción de una base de datos geoespacial.
- Visualización cartográfica mediante mapas interactivos.
- Generación de estadísticas históricas y análisis territoriales.
- Consulta de información mediante filtros y búsquedas especializadas.

Esta integración permite transformar información no estructurada proveniente de múltiples medios de comunicación en una base de conocimiento georreferenciada que facilita el análisis espacial, la consulta histórica y el apoyo a la toma de decisiones.

10.10 Síntesis del Estado del Arte

El análisis del estado del arte evidencia que las tecnologías de monitoreo automatizado, procesamiento de lenguaje natural, georreferenciación y sistemas de información geográfica han alcanzado un alto grado de madurez y ofrecen herramientas eficaces para el tratamiento de grandes volúmenes de información.

No obstante, persiste la necesidad de soluciones que integren estas capacidades en una única plataforma orientada a la clasificación automática de noticias, la generación de información geoespacial y el análisis histórico de incidentes.

En este contexto, **Alert!** representa una propuesta tecnológica que articula procesos de captura automatizada, procesamiento por lotes, clasificación por severidad, georreferenciación y generación de archivos GeoJSON, proporcionando una solución que facilita el monitoreo, el análisis territorial y la construcción de una base de datos geoespacial para apoyar procesos de investigación y toma de decisiones.

Con gusto. A continuación se presenta la **Metodología** para el proyecto **Alert!**. Se propone una metodología basada en **Scrum**, ya que es adecuada para el desarrollo incremental de software y permite adaptar el proyecto a cambios durante su construcción. Esta metodología es coherente con las funcionalidades definidas en los documentos del proyecto, como la captura de noticias, la clasificación automática, la georreferenciación, la generación de archivos GeoJSON y la visualización de incidentes.

11. Metodología

11.1 Enfoque Metodológico

Para el desarrollo de la plataforma **Alert!** se adoptará la metodología ágil **Scrum**, debido a que facilita la planificación, organización y ejecución de proyectos de software mediante ciclos cortos de desarrollo denominados *Sprints*. Este enfoque permite entregar versiones funcionales del sistema de manera incremental, obtener retroalimentación continua y realizar ajustes conforme evolucionen las necesidades del proyecto.

La elección de Scrum responde a la naturaleza del sistema, el cual integra diferentes módulos como la captura automática de noticias, el procesamiento Batch, la clasificación por severidad, la georreferenciación, la generación de archivos GeoJSON y la visualización cartográfica. Cada uno de estos componentes puede desarrollarse, probarse e integrarse progresivamente.

11.2 Metodología Scrum

Scrum es un marco de trabajo ágil basado en la colaboración entre los integrantes del equipo y en la entrega continua de valor mediante iteraciones cortas.

En el proyecto **Alert!**, Scrum permitirá:

- Organizar el desarrollo por funcionalidades.
 - Priorizar los requerimientos de mayor valor.
 - Realizar pruebas continuas.
 - Detectar errores tempranamente.
 - Incorporar mejoras durante el desarrollo.
 - Reducir riesgos asociados a la implementación.
-

11.3 Roles del Proyecto

Durante el desarrollo del sistema se definirán los siguientes roles:

Product Owner

Responsable de definir los requerimientos del proyecto, establecer prioridades y validar que el producto cumpla con las necesidades planteadas.

Entre sus funciones se encuentran:

- Definir el Product Backlog.
 - Priorizar funcionalidades.
 - Aprobar entregables.
 - Validar el cumplimiento de los objetivos.
-

Scrum Master

Responsable de facilitar la aplicación de la metodología Scrum.

Sus principales funciones son:

- Coordinar las reuniones.
 - Eliminar impedimentos.
 - Dar seguimiento al cronograma.
 - Promover las buenas prácticas ágiles.
-

Equipo de Desarrollo

Responsable del diseño, construcción, integración y pruebas del sistema.

Entre sus actividades se encuentran:

- Diseño de la arquitectura.
 - Desarrollo de módulos.
 - Construcción de la base de datos.
 - Desarrollo de interfaces.
 - Integración de componentes.
 - Ejecución de pruebas.
-

11.4 Product Backlog

El Product Backlog estará conformado por las principales funcionalidades del sistema, entre ellas:

- Registro e inicio de sesión de usuarios.
- Captura automática de noticias desde fuentes RSS y APIs.
- Procesamiento Batch de la información.
- Clasificación automática por niveles de severidad.
- Identificación de ubicaciones geográficas.
- Generación de archivos GeoJSON.
- Almacenamiento del historial de noticias.
- Visualización de incidentes en mapas.
- Panel de estadísticas.
- Administración de usuarios y categorías.
- Generación de reportes.
- Publicación de la API de consulta.

Estas funcionalidades corresponden a los requisitos definidos para la plataforma Alert!.

11.5 Planeación de Sprints

El desarrollo del proyecto se organizará en seis Sprints, cada uno con un conjunto de actividades y entregables específicos.

Sprint 1. Análisis y Diseño

Objetivo

Definir los requerimientos y diseñar la arquitectura del sistema.

Actividades

- Levantamiento de requerimientos.
- Diseño de casos de uso.
- Diseño de la arquitectura.
- Diseño del modelo de datos.
- Diseño de la interfaz inicial.

Entregables

- Documento de requerimientos.
 - Diagramas UML.
 - Diseño de base de datos.
 - Arquitectura del sistema.
-

Sprint 2. Captura y Procesamiento Batch

Objetivo

Implementar la captura automática de noticias.

Actividades

- Integración con fuentes RSS.
- Desarrollo del proceso Batch.
- Limpieza y normalización de datos.
- Consolidación de noticias.

Entregables

- Módulo de captura.
- Módulo Batch.
- Base inicial de noticias.

Sprint 3. Clasificación Inteligente

Objetivo

Clasificar automáticamente las noticias.

Actividades

- Análisis del contenido.
- Detección de palabras clave.
- Clasificación por categorías.
- Asignación de niveles de severidad.
- Identificación de entidades geográficas.

Entregables

- Motor de clasificación.
- Clasificador de severidad.
- Módulo de georreferenciación.

Sprint 4. Generación del GeoJSON

Objetivo

Construir la base geoespacial del sistema.

Actividades

- Conversión de datos.
- Generación automática del GeoJSON.
- Validación del formato.
- Integración con la base de datos geográfica.

Entregables

- Archivos GeoJSON.
- Repositorio geoespacial.

Sprint 5. Visualización y Estadísticas

Objetivo

Implementar la interfaz de consulta.

Actividades

- Desarrollo del mapa interactivo.
- Dashboard estadístico.
- Filtros.
- Reportes.
- Historial de eventos.

Entregables

- Portal web funcional.
- Dashboard.
- Reportes.

Sprint 6. Integración y Despliegue

Objetivo

Preparar el sistema para su puesta en funcionamiento.

Actividades

- Integración de módulos.
- Pruebas funcionales.
- Corrección de errores.
- Optimización.
- Despliegue.

Entregables

- Plataforma Alert! operativa.
- Manual técnico.
- Manual de usuario.
- Informe final de pruebas.

11.6 Reuniones Scrum

Durante cada Sprint se realizarán las siguientes reuniones:

- **Sprint Planning:** definición de las actividades del Sprint.
- **Daily Scrum:** reunión diaria para revisar el avance del equipo.
- **Sprint Review:** presentación de los resultados obtenidos.
- **Sprint Retrospective:** evaluación del Sprint para identificar oportunidades de mejora.

Estas reuniones permitirán mantener un seguimiento continuo del proyecto y asegurar la calidad del desarrollo.

11.7 Estrategia de Desarrollo

El desarrollo del sistema seguirá un proceso incremental compuesto por las siguientes etapas:

1. Análisis de requerimientos.
2. Diseño de la arquitectura.
3. Desarrollo de módulos independientes.
4. Integración progresiva de componentes.
5. Pruebas funcionales e integración.
6. Generación de la versión candidata.
7. Despliegue y puesta en funcionamiento.

Este enfoque permitirá validar cada módulo antes de integrarlo con el resto de la plataforma.

11.8 Herramientas de Desarrollo

Para el desarrollo del proyecto se utilizarán herramientas como:

- Python para el procesamiento de datos y la lógica de negocio.
 - Flask para el desarrollo de la aplicación web.
 - HTML, CSS y JavaScript para la interfaz de usuario.
 - MySQL para el almacenamiento de información.
 - GeoJSON como formato de intercambio de datos geográficos.
 - Git y GitHub para el control de versiones.
 - Visual Studio Code como entorno de desarrollo.
 - Bibliotecas para procesamiento de RSS, georreferenciación y visualización cartográfica.
-

11.9 Validación de la Metodología

La aplicación de Scrum permitirá desarrollar la plataforma **Alert!** de forma organizada, incremental y flexible, favoreciendo la integración continua de los módulos de captura de noticias, procesamiento Batch, clasificación automática, generación de archivos GeoJSON y visualización geoespacial. Además, facilitará la detección temprana de errores, la mejora continua del producto y el cumplimiento de los objetivos definidos para el proyecto.

12. Levantamiento de Requisitos

12.1 Introducción

El levantamiento de requisitos constituye una de las etapas fundamentales del proceso de desarrollo de software, ya que permite identificar las necesidades de los usuarios y establecer las funcionalidades que deberá ofrecer el sistema. Esta actividad proporciona la base para el diseño, la implementación y la validación de la solución tecnológica, asegurando que el producto final responda a los objetivos definidos para el proyecto.

En el caso de **Alert!**, el levantamiento de requisitos se realizó a partir del análisis de las necesidades relacionadas con el monitoreo automatizado de noticias, la clasificación de eventos por niveles de severidad, la georreferenciación de incidentes y la generación de información geoespacial para apoyar procesos de consulta, análisis y toma de decisiones. Los requisitos identificados se encuentran documentados en la especificación funcional del proyecto.

12.2 Objetivo del Levantamiento de Requisitos

El propósito del levantamiento de requisitos es definir de manera clara y estructurada las funcionalidades, restricciones y características que deberá cumplir la plataforma **Alert!**, garantizando que el desarrollo del sistema responda a las necesidades de los diferentes tipos de usuarios y facilite el procesamiento automatizado de noticias provenientes de múltiples fuentes de información.

12.3 Técnicas Utilizadas

Para la identificación y documentación de los requisitos del sistema se emplearon las siguientes técnicas:

- Análisis de los procesos asociados al monitoreo y clasificación de noticias.
- Definición de actores y casos de uso del sistema.
- Identificación de funcionalidades requeridas para la captura, procesamiento y consulta de información.
- Elaboración del documento de requisitos funcionales.
- Priorización de funcionalidades de acuerdo con los objetivos del proyecto.

12.4 Actores del Sistema

De acuerdo con la especificación funcional, la plataforma Alert! contempla los siguientes actores:

Usuario Visitante

Puede consultar noticias públicas sin necesidad de autenticarse en el sistema.

Usuario Registrado

Puede personalizar alertas, almacenar noticias favoritas y recibir notificaciones relacionadas con eventos de interés.

Analista

Es responsable de validar las clasificaciones automáticas realizadas por el sistema, modificar niveles de severidad cuando sea necesario y aprobar o rechazar noticias procesadas.

Administrador

Gestiona usuarios, categorías, fuentes de información y la configuración general de la plataforma.

12.5 Necesidades Identificadas

Durante el análisis del proyecto se identificaron las siguientes necesidades:

- Automatizar la recopilación de noticias desde diferentes fuentes RSS y APIs.
 - Reducir el tiempo empleado en la búsqueda manual de información.
 - Clasificar automáticamente los eventos según su nivel de severidad.
 - Identificar la ubicación geográfica de cada incidente.
 - Generar archivos GeoJSON para la construcción de una base de datos geoespacial.
 - Visualizar los incidentes mediante mapas interactivos.
 - Consultar información histórica mediante filtros especializados.
 - Generar estadísticas e indicadores territoriales.
 - Facilitar la administración de usuarios y categorías del sistema.
-

12.6 Requisitos Funcionales Identificados

Como resultado del levantamiento de requisitos se definieron las principales funcionalidades que deberá implementar la plataforma Alert!:

- Registro y administración de usuarios.
- Inicio y cierre seguro de sesión.
- Captura automática de noticias desde fuentes RSS y APIs.
- Clasificación automática de noticias mediante análisis de contenido.
- Clasificación manual por parte de los analistas.
- Gestión de categorías de incidentes.
- Visualización de alertas clasificadas.
- Representación de la severidad mediante código de colores.
- Filtros por país, ciudad, fecha, categoría y nivel de severidad.
- Búsqueda inteligente de noticias.
- Visualización de incidentes sobre mapas interactivos.
- Generación de estadísticas y tendencias.
- Envío de notificaciones para eventos críticos.
- Gestión de favoritos.
- Consulta del historial de eventos.
- Dashboard administrativo.
- Gestión de usuarios.
- Auditoría de modificaciones.
- Generación de reportes.
- Publicación de una API para consulta de información.

12.7 Requerimientos de Información

Para cumplir con sus objetivos, la plataforma deberá procesar y administrar información correspondiente a:

- Título de la noticia.
- Contenido.
- Fecha de publicación.
- Fuente de información.
- Categoría.
- Nivel de severidad.
- Ubicación geográfica.
- Coordenadas.
- Estado de validación.
- Historial de modificaciones.
- Usuario responsable de la validación.

Estos datos conformarán la base para la generación de archivos GeoJSON y el almacenamiento histórico de los incidentes.

12.8 Priorización de Requisitos

Con el fin de organizar el desarrollo del proyecto, los requisitos fueron priorizados según su importancia.

Prioridad Alta

- Captura automática de noticias.
- Procesamiento Batch.
- Clasificación automática.
- Identificación geográfica.
- Generación del GeoJSON.
- Visualización de incidentes.

Prioridad Media

- Dashboard estadístico.
- Reportes.
- Gestión de usuarios.
- Favoritos.
- API pública.

Prioridad Baja

- Personalización de preferencias.
- Funcionalidades complementarias de administración.

12.9 Resultado del Levantamiento de Requisitos

El levantamiento de requisitos permitió establecer de manera clara el conjunto de funcionalidades necesarias para el desarrollo de la plataforma Alert!. La información obtenida constituye la base para el diseño de la arquitectura del sistema, la construcción del modelo de datos, la implementación de los módulos funcionales y la planificación de las actividades de desarrollo mediante la metodología Scrum.

Asimismo, este proceso permitió identificar los actores involucrados, las necesidades de información y las funcionalidades esenciales que harán posible automatizar el monitoreo de noticias, generar archivos GeoJSON y construir una base de datos geoespacial que facilite la consulta, el análisis histórico y la visualización cartográfica de los eventos registrados.

13. Casos de Uso

13.1 Introducción

Los casos de uso describen la interacción entre los diferentes actores y la plataforma **Alert!**, permitiendo identificar las funcionalidades que el sistema debe ofrecer para satisfacer los requerimientos del proyecto. Cada caso de uso representa una secuencia de acciones mediante la cual un usuario o un proceso automatizado logra un objetivo específico dentro de la aplicación.

La definición de estos casos de uso constituye un insumo fundamental para el diseño de la arquitectura del sistema, la implementación de los módulos funcionales y la elaboración de los diagramas UML.

13.2 Actores del Sistema

Los actores identificados para la plataforma son:

- Usuario Visitante
- Usuario Registrado
- Analista
- Administrador
- Sistema Batch

Cada actor interactúa con el sistema de acuerdo con los permisos asignados.

13.3 Diagrama General de Casos de Uso

El sistema Alert! está compuesto por los siguientes procesos principales:

- Consultar noticias.
 - Recibir alertas.
 - Clasificar noticias.
 - Archivar cortes de noticias.
 - Procesar noticias por lotes (Batch).
 - Generar GeoJSON.
 - Realizar seguimiento geográfico.
 - Generar estadísticas.
 - Visualizar mapa de alertas.
 - Administrar usuarios.
-

13.4 Caso de Uso CU-01: Consultar Noticias

Objetivo

Permitir que el usuario consulte las noticias procesadas por la plataforma.

Actor Principal

Usuario.

Precondiciones

- El sistema ha procesado noticias.
- Existen registros almacenados.

Flujo Principal

1. El usuario accede al sistema.
2. Selecciona filtros de búsqueda.
3. El sistema consulta las noticias disponibles.
4. Se muestran las noticias clasificadas.
5. El usuario visualiza el detalle de una noticia.

Postcondición

La información solicitada es presentada al usuario.

Resultado esperado

Consulta rápida de noticias clasificadas por severidad y ubicación.

13.5 Caso de Uso CU-02: Recibir Alerta

Objetivo

Notificar automáticamente a los usuarios cuando ocurra un evento crítico.

Actor Principal

Usuario Registrado.

Precondiciones

- El usuario posee una cuenta activa.
- Existen preferencias de notificación configuradas.

Flujo Principal

1. El usuario configura sus preferencias.
2. El sistema detecta un evento crítico.
3. Se genera la alerta.
4. El usuario recibe la notificación.
5. El usuario consulta la noticia.

Postcondición

La alerta queda registrada en el historial del usuario.

13.6 Caso de Uso CU-03: Clasificar Noticia

Objetivo

Validar y clasificar una noticia según su nivel de severidad.

Actor Principal

Analista.

Precondiciones

- La noticia fue capturada por el proceso Batch.

Flujo Principal

1. El analista revisa la noticia.
2. El sistema presenta la clasificación automática.
3. El analista valida o modifica la severidad.
4. Se publica la clasificación definitiva.

Postcondición

La noticia queda disponible para consulta.

13.7 Caso de Uso CU-04: Archivar Corte de Noticias

Objetivo

Guardar un conjunto de noticias procesadas para su consulta histórica.

Actor Principal

Usuario.

Precondiciones

- Existen noticias procesadas.

Flujo Principal

1. El usuario selecciona la opción Guardar.
2. El sistema genera un archivo con la fecha del corte.
3. El usuario selecciona la ubicación.
4. El sistema crea el archivo persistente.

Postcondición

El corte queda almacenado para futuras consultas.

13.8 Caso de Uso CU-05: Procesamiento Batch y Generación del GeoJSON

Objetivo

Procesar automáticamente las noticias recopiladas y generar un archivo GeoJSON.

Actor Principal

Sistema Batch.

Precondiciones

- Existen fuentes RSS configuradas.
- El proceso Batch está programado.

Flujo Principal

1. El sistema consulta las fuentes RSS.
2. Descarga las noticias nuevas.
3. Elimina duplicados.
4. Normaliza la información.
5. Clasifica cada noticia.
6. Identifica la ubicación geográfica.
7. Genera el archivo GeoJSON.
8. Almacena el documento como base para la base de datos geoespacial.

Postcondición

El GeoJSON queda disponible para mapas, estadísticas y consultas.

13.9 Caso de Uso CU-06: Seguimiento a Ubicación Geográfica

Objetivo

Analizar la distribución geográfica de los eventos registrados.

Actor Principal

Analista.

Flujo Principal

1. Cargar el documento general de noticias.
2. Buscar palabras clave relacionadas con ubicaciones.
3. Generar un documento georreferenciado.
4. Preparar el conjunto de datos.
5. Imputar datos atípicos.
6. Comparar registros geográficos.
7. Analizar la frecuencia de eventos.
8. Contabilizar víctimas.
9. Clasificar la severidad por ubicación.
10. Generar información estadística.

Postcondición

La información geográfica queda preparada para el análisis territorial.

13.10 Caso de Uso CU-07: Generación de Gráficos Estadísticos

Objetivo

Presentar estadísticas sobre las noticias procesadas.

Actor Principal

Analista.

Resultados

- Estadísticas diarias.
 - Estadísticas mensuales.
 - Tendencias.
 - Distribución por severidad.
 - Indicadores geográficos.
-

13.11 Caso de Uso CU-08: Mapa de Alertas

Objetivo

Visualizar sobre un mapa interactivo todos los eventos georreferenciados.

Actor Principal

Analista.

Flujo Principal

1. El usuario abre el mapa.
2. El sistema carga el archivo GeoJSON.
3. Se representan los incidentes mediante marcadores.
4. El usuario consulta el detalle de un evento.
5. Se aplican filtros por fecha, severidad o ubicación.

Postcondición

Los eventos son visualizados geográficamente.

13.12 Restricciones del Sistema

Durante el desarrollo del proyecto deberán cumplirse las siguientes restricciones:

- La severidad únicamente podrá tomar valores entre 1 y 8.
 - Toda noticia deberá pertenecer a una categoría.
 - Toda noticia deberá tener una fuente identificable.
 - El sistema deberá conservar el historial completo de modificaciones.
-

13.13 Criterios de Aceptación

La plataforma será aceptada cuando cumpla los siguientes criterios:

- Clasificación correcta de las noticias.
 - Consulta de información mediante filtros en menos de tres clics.
 - Priorización visual de alertas críticas.
 - Generación de estadísticas consistentes.
 - Conservación del historial completo de noticias e incidentes.
-

13.14 Conclusión

Los casos de uso definidos describen las principales interacciones entre los usuarios y la plataforma **Alert!**, proporcionando una visión funcional del sistema. Estos casos sirven como base para el diseño de los diagramas UML, la implementación de los módulos de software y la validación de los requisitos establecidos para el proyecto. Además, incorporan el proceso Batch como elemento central para la generación del archivo GeoJSON, el cual constituye el insumo principal de la base de datos geoespacial y de las funcionalidades de consulta, análisis y visualización cartográfica.

14. Requerimientos Funcionales

14.1 Introducción

Los requerimientos funcionales describen las funciones y servicios que debe ofrecer la plataforma **Alert!** para satisfacer las necesidades de los usuarios y cumplir con los objetivos del proyecto. Estos requisitos definen el comportamiento esperado del sistema, especificando las operaciones que podrá realizar cada actor y las respuestas que deberá proporcionar la aplicación ante diferentes escenarios de uso.

La identificación de estos requerimientos constituye la base para el diseño de la arquitectura del software, el desarrollo de los módulos funcionales y la ejecución de las pruebas del sistema.

14.2 Gestión de Usuarios

RF-01 Registro de usuarios

El sistema deberá permitir el registro de nuevos usuarios mediante un formulario con validación de datos.

RF-02 Inicio de sesión

El sistema deberá permitir la autenticación segura de los usuarios mediante nombre de usuario o correo electrónico y contraseña.

RF-03 Recuperación de contraseña

El sistema deberá permitir que los usuarios recuperen su contraseña mediante un mecanismo seguro de verificación.

RF-04 Gestión de perfiles

El sistema deberá permitir consultar y actualizar la información del perfil del usuario.

RF-05 Administración de usuarios

El administrador podrá crear, modificar, bloquear, habilitar y eliminar cuentas de usuario.

14.3 Captura de Noticias

RF-06 Configuración de fuentes

El sistema deberá permitir registrar, modificar y eliminar fuentes RSS y APIs de información.

RF-07 Captura automática

El sistema deberá consultar automáticamente las fuentes configuradas para obtener nuevas noticias.

RF-08 Procesamiento Batch

El sistema deberá ejecutar procesos Batch programados para recopilar y procesar la información obtenida desde las diferentes fuentes.

RF-09 Eliminación de duplicados

El sistema deberá identificar y eliminar noticias duplicadas durante el proceso de captura.

RF-10 Normalización

El sistema deberá convertir la información obtenida desde diferentes fuentes a un formato homogéneo para facilitar su procesamiento.

14.4 Clasificación de Noticias

RF-11 Clasificación automática

El sistema deberá analizar automáticamente cada noticia y asignarle una categoría.

RF-12 Clasificación por severidad

El sistema deberá asignar automáticamente un nivel de severidad entre 1 y 8 de acuerdo con el contenido de la noticia.

RF-13 Validación por analista

El analista podrá aceptar o modificar la clasificación generada automáticamente.

RF-14 Gestión de categorías

El administrador podrá crear, modificar y eliminar categorías de incidentes.

14.5 Georreferenciación

RF-15 Identificación geográfica

El sistema deberá identificar automáticamente la ubicación geográfica mencionada en cada noticia.

RF-16 Obtención de coordenadas

El sistema deberá convertir las ubicaciones identificadas en coordenadas geográficas.

RF-17 Generación del GeoJSON

El sistema deberá generar automáticamente un archivo GeoJSON con todos los eventos procesados.

RF-18 Base de datos geoespacial

El sistema deberá almacenar la información georreferenciada para consultas posteriores.

14.6 Consulta de Información

RF-19 Consulta de noticias

Los usuarios podrán consultar noticias mediante filtros.

RF-20 Filtros de búsqueda

El sistema permitirá filtrar noticias por:

- Fecha.
 - País.
 - Ciudad.
 - Categoría.
 - Fuente.
 - Nivel de severidad.
-

RF-21 Búsqueda por palabras clave

El sistema permitirá localizar noticias utilizando palabras clave.

RF-22 Historial

El sistema permitirá consultar el historial completo de noticias procesadas.

14.7 Visualización Cartográfica

RF-23 Mapa interactivo

El sistema deberá visualizar los incidentes sobre un mapa interactivo.

RF-24 Consulta geográfica

El usuario podrá seleccionar un incidente sobre el mapa para visualizar su información detallada.

RF-25 Código de colores

Los incidentes deberán representarse mediante colores asociados a su nivel de severidad.

14.8 Estadísticas

RF-26 Dashboard

El sistema deberá mostrar indicadores estadísticos sobre las noticias procesadas.

RF-27 Gráficos

El sistema deberá generar gráficos relacionados con:

- Número de noticias.
 - Severidad.
 - Categorías.
 - Distribución geográfica.
 - Tendencias temporales.
-

RF-28 Reportes

El sistema permitirá generar reportes de información histórica y estadística.

14.9 Notificaciones

RF-29 Alertas

El sistema deberá generar alertas cuando se detecten eventos críticos.

RF-30 Preferencias

El usuario podrá configurar las categorías sobre las cuales desea recibir notificaciones.

14.10 Administración

RF-31 Gestión de fuentes

El administrador podrá administrar las fuentes de información utilizadas por el sistema.

RF-32 Gestión de categorías

El administrador podrá administrar las categorías utilizadas para clasificar las noticias.

RF-33 Auditoría

El sistema deberá registrar todas las modificaciones realizadas por los usuarios administrativos.

RF-34 Configuración

El administrador podrá modificar los parámetros generales del sistema.

14.11 API del Sistema

RF-35 API REST

El sistema deberá publicar una API REST que permita consultar noticias, estadísticas e incidentes georreferenciados.

RF-36 Exportación

El sistema permitirá exportar la información en formatos como:

- GeoJSON.
 - JSON.
 - CSV.
 - PDF.
-

14.12 Matriz Resumen de Requerimientos Funcionales

Código	Requerimiento	Actor
RF-01	Registro de usuarios	Usuario
RF-02	Inicio de sesión	Usuario
RF-03	Recuperación de contraseña	Usuario
RF-04	Gestión de perfiles	Usuario
RF-05	Administración de usuarios	Administrador
RF-06	Gestión de fuentes RSS	Administrador
RF-07	Captura automática de noticias	Sistema Batch
RF-08	Procesamiento Batch	Sistema Batch
RF-09	Eliminación de duplicados	Sistema Batch
RF-10	Normalización de datos	Sistema Batch
RF-11	Clasificación automática	Sistema
RF-12	Clasificación por severidad	Sistema
RF-13	Validación de clasificación	Analista
RF-14	Gestión de categorías	Administrador
RF-15	Identificación geográfica	Sistema
RF-16	Obtención de coordenadas	Sistema
RF-17	Generación de GeoJSON	Sistema

Código	Requerimiento	Actor
RF-18	Base de datos geoespacial	Sistema
RF-19	Consulta de noticias	Usuario
RF-20	Filtros de búsqueda	Usuario
RF-21	Búsqueda por palabras clave	Usuario
RF-22	Historial de noticias	Usuario
RF-23	Mapa interactivo	Usuario
RF-24	Consulta geográfica	Usuario
RF-25	Visualización por colores	Sistema
RF-26	Dashboard estadístico	Usuario
RF-27	Gráficos estadísticos	Usuario
RF-28	Reportes	Usuario
RF-29	Generación de alertas	Sistema
RF-30	Configuración de notificaciones	Usuario
RF-31	Administración de fuentes	Administrador
RF-32	Administración de categorías	Administrador
RF-33	Auditoría	Administrador
RF-34	Configuración del sistema	Administrador
RF-35	API REST	Sistema
RF-36	Exportación de datos	Usuario

14.13 Conclusión

Los requerimientos funcionales definidos establecen el conjunto de capacidades que debe ofrecer la plataforma **Alert!** para cumplir con los objetivos del proyecto. Su implementación permitirá automatizar la captura y el procesamiento de noticias mediante procesos Batch, clasificar los eventos según su nivel de severidad, georreferenciarlos, generar archivos GeoJSON y proporcionar herramientas de consulta, análisis estadístico y visualización cartográfica. Estos requisitos constituyen la base para el diseño, desarrollo, pruebas y validación del sistema.

Con gusto. A continuación se presenta el capítulo **Requerimientos No Funcionales** para el proyecto **Alert!**. Los documentos que compartiste se centran principalmente en los requisitos funcionales y casos de uso; por ello, los requerimientos no funcionales que siguen se formulan a partir de las características técnicas esperadas para una plataforma de este tipo, complementando la especificación funcional del proyecto.

15. Requerimientos No Funcionales

15.1 Introducción

Los requerimientos no funcionales establecen las características de calidad que debe cumplir la plataforma **Alert!**, definiendo aspectos relacionados con el rendimiento, la seguridad, la disponibilidad, la confiabilidad, la usabilidad, la escalabilidad y la mantenibilidad del sistema. Estos requisitos no describen funcionalidades específicas, sino las condiciones bajo las cuales dichas funcionalidades deben operar para garantizar un servicio eficiente y confiable.

15.2 Rendimiento

RNF-01 Tiempo de respuesta

El sistema deberá responder a las consultas realizadas por los usuarios en un tiempo promedio inferior a **3 segundos**, siempre que la carga del servidor se encuentre dentro de los parámetros normales de operación.

RNF-02 Procesamiento Batch

El proceso Batch deberá ejecutarse automáticamente en los intervalos configurados, procesando todas las noticias disponibles sin afectar el funcionamiento de la aplicación web.

RNF-03 Capacidad de procesamiento

La plataforma deberá soportar el procesamiento simultáneo de noticias provenientes de múltiples fuentes RSS y APIs, garantizando la integridad de la información.

15.3 Disponibilidad

RNF-04 Disponibilidad del servicio

La plataforma deberá mantener una disponibilidad mínima del **99 %** durante el horario de operación definido para el proyecto.

RNF-05 Recuperación ante fallos

El sistema deberá registrar los errores ocurridos durante el procesamiento Batch y permitir la recuperación automática o manual del proceso sin pérdida de información.

15.4 Seguridad

RNF-06 Autenticación

El acceso a las funciones administrativas deberá realizarse mediante autenticación de usuarios y contraseñas seguras.

RNF-07 Control de acceso

El sistema deberá implementar control de acceso basado en roles (Administrador, Analista, Usuario Registrado y Usuario Visitante), limitando las funcionalidades disponibles según el perfil del usuario.

RNF-08 Protección de datos

La información almacenada deberá protegerse mediante mecanismos que garanticen la confidencialidad, integridad y disponibilidad de los datos.

RNF-09 Registro de auditoría

El sistema deberá registrar las acciones realizadas por los usuarios con privilegios administrativos para facilitar procesos de seguimiento y control.

15.5 Usabilidad

RNF-10 Interfaz amigable

La aplicación deberá ofrecer una interfaz intuitiva, organizada y de fácil navegación, permitiendo a los usuarios acceder rápidamente a las funcionalidades principales.

RNF-11 Adaptabilidad

La interfaz deberá visualizarse correctamente en los principales navegadores web y adaptarse a diferentes resoluciones de pantalla.

RNF-12 Facilidad de aprendizaje

Un usuario con conocimientos básicos en informática deberá ser capaz de utilizar las funciones principales del sistema después de una breve inducción.

15.6 Confiabilidad

RNF-13 Integridad de la información

El sistema deberá garantizar que las noticias procesadas no sean alteradas durante las etapas de captura, clasificación y almacenamiento.

RNF-14 Consistencia de los datos

Los registros almacenados en la base de datos deberán mantenerse consistentes durante las operaciones de inserción, actualización y consulta.

15.7 Escalabilidad

RNF-15 Crecimiento del sistema

La arquitectura deberá permitir la incorporación de nuevas fuentes de información, categorías de incidentes y funcionalidades sin requerir modificaciones significativas en la estructura del software.

RNF-16 Incremento de usuarios

La plataforma deberá soportar el crecimiento progresivo del número de usuarios y del volumen de noticias procesadas.

15.8 Mantenibilidad

RNF-17 Modularidad

El software deberá desarrollarse mediante una arquitectura modular que facilite la actualización y reutilización de componentes.

RNF-18 Documentación

El proyecto deberá contar con documentación técnica y funcional que facilite el mantenimiento y evolución del sistema.

RNF-19 Control de versiones

El código fuente deberá administrarse mediante un sistema de control de versiones para garantizar la trazabilidad de los cambios realizados durante el desarrollo.

15.9 Portabilidad

RNF-20 Compatibilidad

La aplicación deberá ejecutarse en sistemas operativos compatibles con Python y ser accesible desde los principales navegadores web modernos.

RNF-21 Despliegue

El sistema deberá poder desplegarse en servidores locales o servicios de alojamiento en la nube, permitiendo su instalación en diferentes entornos.

15.10 Interoperabilidad

RNF-22 Integración

La plataforma deberá integrarse con fuentes RSS, APIs y herramientas cartográficas mediante protocolos y formatos de intercambio estandarizados.

RNF-23 Formatos de intercambio

El sistema deberá generar y consumir formatos abiertos como JSON, GeoJSON y CSV para facilitar la interoperabilidad con otras aplicaciones.

15.11 Calidad del Software

RNF-24 Cumplimiento de estándares

El desarrollo del proyecto deberá seguir buenas prácticas de ingeniería de software y considerar estándares internacionales como ISO/IEC 25010 para la calidad del software, ISO/IEC 12207 para los procesos del ciclo de vida e ISO/IEC 29119 para las pruebas de software.

RNF-25 Pruebas

Antes de su puesta en producción, el sistema deberá superar pruebas funcionales, de integración, rendimiento y aceptación por parte de los usuarios.

15.12 Respaldo y Recuperación

RNF-26 Copias de seguridad

El sistema deberá permitir realizar copias de seguridad periódicas de la base de datos y de los archivos GeoJSON generados por los procesos Batch.

RNF-27 Recuperación de información

La plataforma deberá disponer de mecanismos para restaurar la información almacenada en caso de fallos del sistema o pérdida de datos.

15.13 Matriz de Requerimientos No Funcionales

Código	Requerimiento	Categoría
RNF-01	Tiempo de respuesta	Rendimiento
RNF-02	Procesamiento Batch programado	Rendimiento
RNF-03	Capacidad de procesamiento	Rendimiento
RNF-04	Disponibilidad del servicio	Disponibilidad
RNF-05	Recuperación ante fallos	Disponibilidad
RNF-06	Autenticación segura	Seguridad
RNF-07	Control de acceso por roles	Seguridad
RNF-08	Protección de datos	Seguridad
RNF-09	Auditoría de acciones	Seguridad
RNF-10	Interfaz amigable	Usabilidad
RNF-11	Adaptabilidad a navegadores	Usabilidad
RNF-12	Facilidad de aprendizaje	Usabilidad
RNF-13	Integridad de la información	Confiabilidad
RNF-14	Consistencia de datos	Confiabilidad
RNF-15	Escalabilidad de la arquitectura	Escalabilidad
RNF-16	Soporte para crecimiento	Escalabilidad
RNF-17	Arquitectura modular	Mantenibilidad
RNF-18	Documentación técnica	Mantenibilidad
RNF-19	Control de versiones	Mantenibilidad
RNF-20	Compatibilidad multiplataforma	Portabilidad
RNF-21	Despliegue flexible	Portabilidad
RNF-22	Integración con servicios externos	Interoperabilidad
RNF-23	Uso de formatos abiertos	Interoperabilidad

Código	Requerimiento	Categoría
RNF-24	Cumplimiento de estándares de calidad	Calidad
RNF-25	Pruebas del sistema	Calidad
RNF-26	Copias de seguridad	Respaldo
RNF-27	Recuperación de información	Respaldo

15.14 Conclusión

Los requerimientos no funcionales definidos establecen los criterios de calidad que deberá cumplir la plataforma **Alert!** para garantizar un funcionamiento seguro, eficiente y confiable. Estos requisitos complementan los requerimientos funcionales al definir condiciones relacionadas con el rendimiento, la seguridad, la disponibilidad, la escalabilidad, la mantenibilidad y la interoperabilidad, proporcionando una base sólida para el diseño de la arquitectura, el desarrollo de la aplicación y su futura operación en entornos reales.

Con gusto. A continuación se presenta el capítulo **Arquitectura del Sistema** para el proyecto **Alert!**. El contenido está alineado con los requisitos funcionales y casos de uso del proyecto, que describen una plataforma compuesta por módulos para la captura de noticias, procesamiento Batch, clasificación automática, georreferenciación, generación de archivos GeoJSON, base de datos geoespacial y visualización mediante mapas.

16. Arquitectura del Sistema

16.1 Introducción

La arquitectura del sistema define la estructura general de la plataforma **Alert!**, identificando los componentes principales, sus responsabilidades y la forma en que interactúan para cumplir los objetivos del proyecto. Una arquitectura bien definida facilita el desarrollo, la integración, el mantenimiento y la escalabilidad del software, además de permitir la incorporación de nuevas funcionalidades en futuras versiones.

La plataforma ha sido diseñada bajo una arquitectura modular de tipo **Cliente–Servidor**, complementada con un proceso de **procesamiento por lotes (Batch)** encargado de recopilar, analizar y transformar la información obtenida desde múltiples fuentes de noticias.

16.2 Tipo de Arquitectura

El proyecto **Alert!** adopta una **arquitectura Cliente–Servidor multicapa**, organizada en cinco capas principales:

1. Capa de Presentación.
2. Capa de Servicios Web.
3. Capa de Lógica de Negocio.
4. Capa de Procesamiento Batch.
5. Capa de Persistencia de Datos.

Esta organización permite separar las responsabilidades de cada componente, favoreciendo la reutilización del código, la mantenibilidad y la escalabilidad de la aplicación.

16.3 Arquitectura General

La plataforma está conformada por los siguientes componentes:

Cliente Web

Corresponde a la interfaz utilizada por los usuarios para interactuar con la plataforma.

Sus funciones incluyen:

- Inicio de sesión.
- Consulta de noticias.
- Visualización del mapa.
- Consulta de estadísticas.
- Administración del sistema.
- Configuración de alertas.

La interfaz será accesible mediante navegadores web modernos.

Servidor Web

El servidor ejecuta la aplicación desarrollada en **Python** utilizando **Flask** como framework principal.

Entre sus responsabilidades se encuentran:

- Procesar solicitudes HTTP.
- Gestionar sesiones de usuario.
- Ejecutar reglas de negocio.

- Consultar la base de datos.
 - Publicar la API REST.
 - Administrar la autenticación.
-

Módulo Batch

Es el componente central del proyecto.

Su función consiste en ejecutar automáticamente procesos programados para:

- Consultar fuentes RSS.
- Descargar noticias.
- Eliminar duplicados.
- Normalizar la información.
- Clasificar eventos.
- Detectar ubicaciones.
- Generar archivos GeoJSON.
- Actualizar la base de datos.

Este proceso puede ejecutarse mediante tareas programadas utilizando cron (Linux) o el Programador de tareas de Windows.

Motor de Clasificación

Este componente analiza el contenido de cada noticia utilizando reglas de negocio y técnicas de procesamiento de texto.

Sus funciones son:

- Detectar palabras clave.
 - Identificar categorías.
 - Asignar niveles de severidad.
 - Preparar la información para el análisis geográfico.
-

Módulo de Georreferenciación

Permite transformar los nombres de ciudades, municipios o países identificados en las noticias en coordenadas geográficas.

Como resultado produce:

- Latitud.
- Longitud.
- País.
- Departamento.

- Ciudad.

Esta información será utilizada posteriormente para la generación del archivo GeoJSON.

Generador GeoJSON

Este módulo convierte la información procesada en un documento GeoJSON compatible con Sistemas de Información Geográfica (SIG).

Cada registro contendrá información como:

- Identificador.
- Fecha.
- Fuente.
- Categoría.
- Nivel de severidad.
- Coordenadas.
- Descripción del evento.

El archivo generado constituye la fuente principal para la base de datos geoespacial.

Base de Datos

La plataforma utilizará una base de datos relacional (MySQL) para almacenar:

- Usuarios.
- Noticias.
- Categorías.
- Fuentes RSS.
- Historial de eventos.
- Configuración del sistema.
- Registros de auditoría.

La información geográfica podrá almacenarse directamente o derivarse del archivo GeoJSON para facilitar consultas y análisis espaciales.

API REST

La aplicación expondrá una API REST que permitirá consultar:

- Noticias.
- Estadísticas.
- Categorías.
- Incidentes.

- Información geográfica.
- Alertas.

Esto facilitará la integración con aplicaciones externas.

16.4 Flujo General del Sistema

El funcionamiento general de la plataforma se desarrolla mediante las siguientes etapas:

1. El proceso Batch consulta las fuentes RSS configuradas.
 2. Se descargan las noticias publicadas.
 3. El sistema elimina registros duplicados.
 4. Se normaliza la información.
 5. El motor de clasificación analiza cada noticia.
 6. Se determina la categoría y el nivel de severidad.
 7. El módulo de georreferenciación identifica la ubicación del evento.
 8. Se obtienen las coordenadas geográficas.
 9. Se genera automáticamente el archivo GeoJSON.
 10. La información se almacena en la base de datos.
 11. El servidor publica los datos mediante la API.
 12. La interfaz web presenta mapas, estadísticas y reportes a los usuarios.
-

16.5 Arquitectura por Capas

Capa de Presentación

Responsable de la interacción con el usuario.

Incluye:

- Portal web.
 - Dashboard.
 - Mapa interactivo.
 - Formularios.
 - Reportes.
-

Capa de Servicios

Gestiona la comunicación entre la interfaz y la lógica del negocio.

Incluye:

- API REST.
 - Autenticación.
 - Gestión de sesiones.
 - Control de acceso.
-

Capa de Negocio

Implementa las reglas funcionales del sistema.

Incluye:

- Clasificación.
 - Gestión de noticias.
 - Gestión de usuarios.
 - Estadísticas.
 - Reportes.
-

Capa Batch

Realiza el procesamiento automatizado de noticias.

Incluye:

- Captura RSS.
 - Limpieza.
 - Normalización.
 - Clasificación.
 - Georreferenciación.
 - Generación del GeoJSON.
-

Capa de Datos

Responsable del almacenamiento permanente.

Incluye:

- Base de datos MySQL.
 - Archivos GeoJSON.
 - Registros históricos.
 - Configuración.
 - Auditoría.
-

16.6 Tecnologías Utilizadas

La arquitectura propuesta contempla el uso de las siguientes tecnologías:

Componente	Tecnología
Lenguaje de programación	Python
Framework Backend	Flask
Frontend	HTML5, CSS3, JavaScript
Base de datos	MySQL
Formato geográfico	GeoJSON
Servicios cartográficos	Leaflet u OpenLayers
Consumo RSS	Feedparser
Procesamiento de datos	Pandas
Procesamiento de texto	spaCy y expresiones regulares
Georreferenciación	Geopy (Nominatim) u otro servicio compatible
API	REST
Control de versiones	Git y GitHub

16.7 Principios de Diseño

La arquitectura del sistema se fundamenta en los siguientes principios:

- Modularidad.
- Bajo acoplamiento.
- Alta cohesión.
- Escalabilidad.
- Reutilización de componentes.
- Separación de responsabilidades.
- Facilidad de mantenimiento.
- Seguridad.
- Integridad de los datos.

Estos principios permiten que la plataforma pueda evolucionar sin afectar significativamente los componentes existentes.

16.8 Escalabilidad

La arquitectura permite incorporar nuevas funcionalidades sin modificar la estructura principal del sistema.

Entre las posibles ampliaciones se encuentran:

- Nuevas fuentes RSS.
 - Integración con APIs adicionales.
 - Nuevos algoritmos de clasificación.
 - Modelos de Inteligencia Artificial.
 - Nuevos formatos de exportación.
 - Integración con plataformas SIG.
 - Aplicaciones móviles.
 - Procesamiento en tiempo real.
-

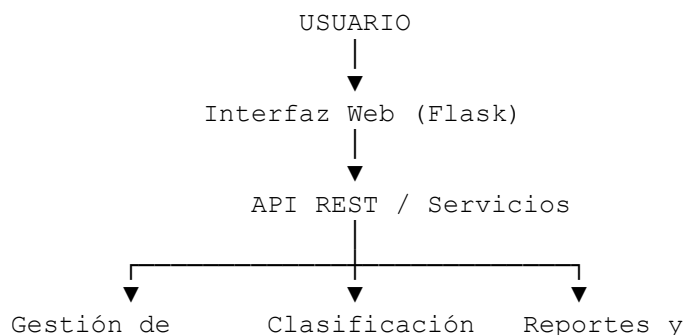
16.9 Seguridad

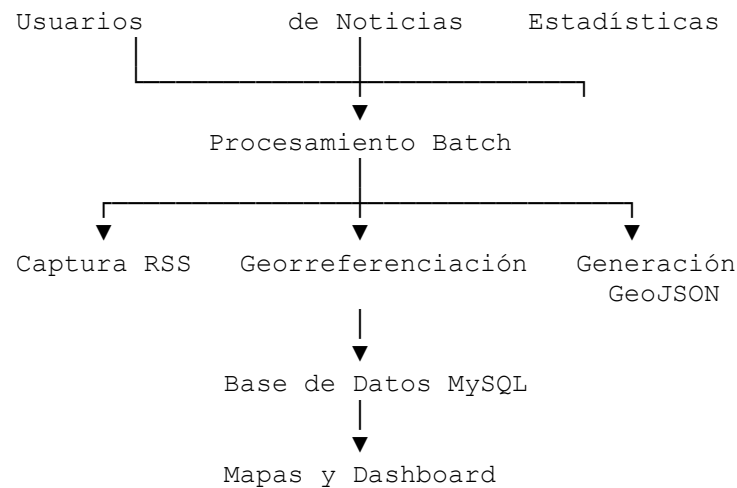
La arquitectura contempla mecanismos para proteger la información procesada mediante:

- Autenticación de usuarios.
 - Control de acceso basado en roles.
 - Registro de auditoría.
 - Validación de entradas.
 - Copias de seguridad.
 - Protección de credenciales.
 - Comunicación segura mediante HTTPS en ambientes de producción.
-

16.10 Diagrama Conceptual de la Arquitectura

La arquitectura puede representarse de manera simplificada mediante el siguiente esquema:





16.11 Beneficios de la Arquitectura

La arquitectura propuesta ofrece las siguientes ventajas:

- Automatización del procesamiento de noticias.
- Separación clara de responsabilidades.
- Facilidad para incorporar nuevos módulos.
- Escalabilidad para aumentar el volumen de datos y usuarios.
- Integración con sistemas externos mediante API REST.
- Compatibilidad con herramientas SIG gracias al uso de GeoJSON.
- Mayor mantenibilidad y reutilización del código.
- Mejor desempeño al desacoplar el procesamiento Batch de la interfaz web.

16.12 Conclusión

La arquitectura diseñada para **Alert!** proporciona una estructura modular, escalable y mantenible que integra la captura automática de noticias, el procesamiento Batch, la clasificación por severidad, la georreferenciación, la generación de archivos GeoJSON y la visualización cartográfica. Esta organización facilita el desarrollo incremental del proyecto, la integración de nuevas funcionalidades y el procesamiento eficiente de grandes volúmenes de información, cumpliendo con los objetivos planteados para la plataforma.

Como complemento a este capítulo, es recomendable incluir en el documento final los siguientes diagramas UML y de arquitectura:

1. **Diagrama de Arquitectura Cliente–Servidor.**
2. **Diagrama de Componentes UML.**
3. **Diagrama de Despliegue (Deployment Diagram).**
4. **Diagrama de Secuencia del proceso Batch.**
5. **Diagrama de Flujo de Datos (DFD).**

6. Diagrama de Arquitectura de Base de Datos (ERD).

Estos diagramas fortalecerán la documentación técnica del proyecto y son habituales en proyectos de grado y formulaciones de software del SENA.

17. Tecnologías Utilizadas

17.1 Introducción

El desarrollo de la plataforma **Alert!** requiere la integración de diversas tecnologías de software orientadas al procesamiento automatizado de información, análisis de noticias, georreferenciación y visualización de datos geoespaciales. La selección de estas herramientas responde a criterios de eficiencia, interoperabilidad, escalabilidad y disponibilidad de software libre, permitiendo construir una solución robusta para el monitoreo y análisis de información proveniente de múltiples fuentes.

La arquitectura tecnológica del proyecto se apoya en un proceso **Batch** encargado de recopilar y procesar noticias, una aplicación web desarrollada en Python y un conjunto de herramientas para la generación de información geográfica en formato GeoJSON.

17.2 Lenguaje de Programación

Python

El núcleo del sistema fue desarrollado utilizando **Python**, debido a su versatilidad, facilidad de integración con bibliotecas de procesamiento de datos y amplia adopción en proyectos de inteligencia artificial y análisis de información.

Python se utiliza para:

- Implementar la lógica de negocio.
 - Ejecutar el procesamiento Batch.
 - Consumir fuentes RSS.
 - Clasificar noticias.
 - Generar archivos GeoJSON.
 - Gestionar la comunicación con la base de datos.
 - Exponer servicios mediante API REST.
-

17.3 Framework Web

Flask

La aplicación web utiliza **Flask** como framework para el desarrollo del backend.

Flask proporciona:

- Gestión de rutas HTTP.
- Administración de sesiones.
- Integración con plantillas HTML.
- Desarrollo de servicios REST.
- Facilidad para integrar módulos independientes.

Su arquitectura ligera facilita el mantenimiento y la evolución del sistema.

17.4 Procesamiento Batch

Uno de los componentes principales de la plataforma es el **proceso Batch**, encargado de automatizar la captura y transformación de la información.

Sus principales funciones son:

- Consulta periódica de fuentes RSS.
- Descarga de noticias.
- Eliminación de registros duplicados.
- Limpieza y normalización de datos.
- Clasificación automática de eventos.
- Identificación de ubicaciones geográficas.
- Generación del archivo GeoJSON.
- Actualización de la base de datos.

Este procesamiento desacopla la captura de información de la interfaz web, mejorando el rendimiento del sistema y permitiendo procesar grandes volúmenes de datos sin afectar la experiencia del usuario.

17.5 Base de Datos

MySQL

Para el almacenamiento permanente de la información se utiliza **MySQL**, un sistema de gestión de bases de datos relacional ampliamente utilizado en aplicaciones web.

La base de datos almacena:

- Usuarios.
- Noticias procesadas.
- Categorías.
- Fuentes RSS.
- Coordenadas geográficas.

- Historial de eventos.
- Configuración del sistema.
- Registros de auditoría.

Su estructura relacional garantiza la integridad y consistencia de los datos.

17.6 Procesamiento de Noticias

Para la captura y análisis de noticias se emplean bibliotecas especializadas que permiten automatizar el tratamiento de la información.

Entre ellas se destacan:

- **Feedparser**, para consumir canales RSS.
- **Requests**, para realizar solicitudes HTTP.
- **BeautifulSoup**, para procesar contenido HTML cuando sea necesario.
- **Pandas**, para la limpieza, transformación y análisis de los datos.

Estas herramientas facilitan la consolidación de información procedente de múltiples fuentes.

17.7 Procesamiento del Lenguaje Natural

El sistema incorpora herramientas de Procesamiento de Lenguaje Natural (NLP) para identificar información relevante dentro del contenido de las noticias.

Entre las funciones implementadas se encuentran:

- Detección de palabras clave.
- Reconocimiento de entidades geográficas.
- Clasificación temática.
- Identificación de niveles de severidad.
- Preparación de datos para la georreferenciación.

Para estas tareas pueden emplearse bibliotecas como **spaCy** y expresiones regulares, dependiendo de la complejidad del análisis requerido.

17.8 Georreferenciación

El proyecto utiliza servicios de geocodificación para convertir nombres de ciudades, municipios y países en coordenadas geográficas.

Estas coordenadas permiten:

- Representar eventos sobre mapas.
- Generar estadísticas territoriales.
- Construir la base de datos geoespacial.
- Elaborar el archivo GeoJSON.

Para este propósito se utilizan bibliotecas como **Geopy** y servicios compatibles con **Nominatim (OpenStreetMap)**.

17.9 Formato GeoJSON

La información geográfica generada por el sistema se almacena utilizando el estándar **GeoJSON**, un formato basado en JSON ampliamente utilizado en Sistemas de Información Geográfica (SIG).

Cada elemento GeoJSON contiene información como:

- Identificador del evento.
- Ubicación geográfica.
- Coordenadas.
- Categoría.
- Nivel de severidad.
- Fecha de publicación.
- Fuente.
- Descripción de la noticia.

Este formato facilita la interoperabilidad con plataformas cartográficas y herramientas de análisis geoespacial.

17.10 Visualización Cartográfica

Para la representación gráfica de los eventos georreferenciados se emplean bibliotecas JavaScript compatibles con GeoJSON, tales como:

- Leaflet.
- OpenLayers.

Estas herramientas permiten:

- Visualizar incidentes sobre mapas interactivos.
- Aplicar filtros geográficos.
- Consultar el detalle de cada evento.
- Representar niveles de severidad mediante códigos de colores.

17.11 Desarrollo de la Interfaz

La interfaz gráfica del sistema se desarrolla utilizando tecnologías web estándar:

- HTML5.
- CSS3.
- JavaScript.

Estas tecnologías permiten construir una aplicación accesible desde navegadores modernos y adaptable a diferentes resoluciones de pantalla.

17.12 Control de Versiones

Durante el desarrollo del proyecto se utiliza **Git** como sistema de control de versiones y **GitHub** como repositorio para el almacenamiento del código fuente.

Estas herramientas permiten:

- Registrar cambios.
 - Trabajar de forma colaborativa.
 - Recuperar versiones anteriores.
 - Gestionar ramas de desarrollo.
 - Facilitar el mantenimiento del software.
-

17.13 Entorno de Desarrollo

Para la construcción de la plataforma se emplean herramientas como:

- Visual Studio Code.
- Python Virtual Environment (venv).
- Git.
- Navegadores web para pruebas.
- Servidor local Flask.

Estas herramientas facilitan la programación, depuración y validación del sistema.

17.14 Tecnologías Complementarias

Durante el desarrollo también se emplean tecnologías adicionales para mejorar la calidad del software:

- JSON para intercambio de información.
- REST para la comunicación entre servicios.
- HTTPS para comunicaciones seguras en producción.
- CSV para exportación de datos.
- API de geocodificación para obtener coordenadas geográficas.

17.15 Resumen de Tecnologías

Componente	Tecnología	Función
Lenguaje de programación	Python	Lógica del sistema y procesamiento Batch
Framework Backend	Flask	Desarrollo de la aplicación web
Base de datos	MySQL	Almacenamiento de la información
Procesamiento RSS	Feedparser	Captura automática de noticias
Solicitudes HTTP	Requests	Consumo de servicios web
Procesamiento HTML	BeautifulSoup	Extracción de contenido
Análisis de datos	Pandas	Limpieza y transformación de datos
Procesamiento NLP	spaCy	Clasificación e identificación de entidades
Georreferenciación	Geopy / Nominatim	Obtención de coordenadas
Formato geográfico	GeoJSON	Intercambio de información espacial
Mapas interactivos	Leaflet u OpenLayers	Visualización cartográfica
Frontend	HTML5, CSS3, JavaScript	Interfaz de usuario
Control de versiones	Git y GitHub	Gestión del código fuente
IDE	Visual Studio Code	Desarrollo y depuración

17.16 Beneficios de las Tecnologías Seleccionadas

La combinación de estas tecnologías ofrece múltiples ventajas para el proyecto **Alert!**, entre ellas:

- Automatización del monitoreo de noticias.
 - Procesamiento eficiente de grandes volúmenes de información.
 - Integración con múltiples fuentes RSS y APIs.
 - Generación automática de archivos GeoJSON.
 - Visualización geográfica interactiva.
 - Escalabilidad para incorporar nuevas funcionalidades.
 - Uso de estándares abiertos que favorecen la interoperabilidad.
 - Facilidad de mantenimiento y evolución del software.
-

17.17 Conclusión

Las tecnologías seleccionadas para el desarrollo de **Alert!** proporcionan una infraestructura sólida para la construcción de una plataforma de monitoreo automatizado de noticias. La integración de Python, Flask, MySQL, procesamiento Batch, herramientas de procesamiento de lenguaje natural y tecnologías geoespaciales permite implementar un sistema capaz de recopilar, analizar, clasificar y representar información georreferenciada de forma eficiente, garantizando la escalabilidad, mantenibilidad e interoperabilidad de la solución.

Con gusto. A continuación se presenta el capítulo **Diseño de la Base de Datos** para el proyecto **Alert!**. Este documento se basa en la arquitectura, los casos de uso y los requerimientos funcionales del proyecto, donde se establece que la plataforma almacena noticias, usuarios, fuentes RSS, categorías, coordenadas geográficas y archivos GeoJSON como parte del proceso Batch.

18. Diseño de la Base de Datos

18.1 Introducción

La base de datos constituye uno de los componentes fundamentales de la plataforma **Alert!**, ya que permite almacenar de manera estructurada toda la información generada durante el monitoreo automatizado de noticias. Su diseño garantiza la organización, integridad y disponibilidad de los datos requeridos para la clasificación de eventos, la georreferenciación, la generación de archivos GeoJSON y la consulta histórica de incidentes.

El modelo de datos fue diseñado bajo el paradigma relacional utilizando **MySQL** como Sistema Gestor de Bases de Datos (SGBD), permitiendo mantener relaciones consistentes entre las diferentes entidades que conforman la plataforma.

18.2 Objetivos del Diseño

El diseño de la base de datos tiene como objetivos:

- Almacenar de forma organizada la información capturada desde múltiples fuentes.
 - Garantizar la integridad y consistencia de los datos.
 - Evitar redundancia mediante la normalización de las tablas.
 - Facilitar consultas rápidas sobre noticias e incidentes.
 - Soportar la generación de archivos GeoJSON.
 - Mantener el historial de eventos procesados.
 - Permitir la administración de usuarios y configuraciones del sistema.
-

18.3 Modelo de Datos

El modelo relacional de Alert! está compuesto por las siguientes entidades principales:

- Usuarios
- Roles
- Fuentes RSS
- Categorías
- Noticias
- Ubicaciones
- Eventos Geográficos
- Alertas
- Historial de Procesamiento Batch
- Auditoría

Estas entidades se relacionan para garantizar un flujo coherente de la información desde la captura de noticias hasta su consulta en el mapa interactivo.

18.4 Descripción de las Entidades

Tabla: Roles

Almacena los perfiles de acceso del sistema.

Campos principales

- id_rol
- nombre
- descripción

Relación:

- Un rol puede estar asociado a varios usuarios.

Tabla: Usuarios

Contiene la información de los usuarios registrados en la plataforma.

Campos principales

- id_usuario
- nombre
- apellido
- correo electrónico
- contraseña
- estado
- fecha de creación
- id_rol

Relación:

- Un usuario pertenece a un rol.
- Un usuario puede generar registros de auditoría.

Tabla: Fuentes_RSS

Almacena las fuentes de información utilizadas por el proceso Batch.

Campos principales

- id_fuente
- nombre
- URL
- país
- estado
- fecha de registro

Relación:

- Una fuente genera múltiples noticias.

Tabla: Categorías

Contiene las categorías utilizadas para clasificar los eventos.

Ejemplos:

- Seguridad.
- Desastre natural.
- Salud.
- Orden público.
- Medio ambiente.
- Tecnología.

Campos

- id_categoria
- nombre
- descripción

Tabla: Noticias

Es la entidad principal del sistema.

Almacena toda la información procesada por el Batch.

Campos principales

- id_noticia
- título
- descripción
- contenido
- fecha_publicación
- fecha_procesamiento
- severidad
- estado
- id_categoria
- id_fuente
- id_ubicación

Relaciones:

- Una noticia pertenece a una categoría.
 - Una noticia proviene de una fuente RSS.
 - Una noticia posee una ubicación geográfica.
-

Tabla: Ubicaciones

Almacena la información geográfica obtenida durante el análisis de las noticias.

Campos

- id_ubicación
- país
- departamento
- ciudad
- latitud
- longitud

Relación:

- Una ubicación puede asociarse a varias noticias.
-

Tabla: Eventos_GeoJSON

Representa la información utilizada para generar los mapas.

Campos

- id_evento
- id_noticia
- geometría
- tipo_geometría
- propiedades
- fecha_generación

Esta tabla puede almacenar directamente el contenido geográfico o la referencia al archivo GeoJSON generado por el proceso Batch.

Tabla: Alertas

Contiene las notificaciones enviadas a los usuarios.

Campos

- id_alerta
 - id_usuario
 - id_noticia
 - fecha_envío
 - estado
-

Tabla: Batch_Log

Registra la ejecución de cada proceso Batch.

Campos

- id_batch
- fecha_inicio
- fecha_fin
- noticias_procesadas
- errores
- estado

Permite realizar seguimiento al funcionamiento del sistema.

Tabla: Auditoría

Registra todas las acciones administrativas.

Campos

- id_auditoría
 - usuario
 - acción
 - tabla_afectada
 - fecha
 - dirección IP
-

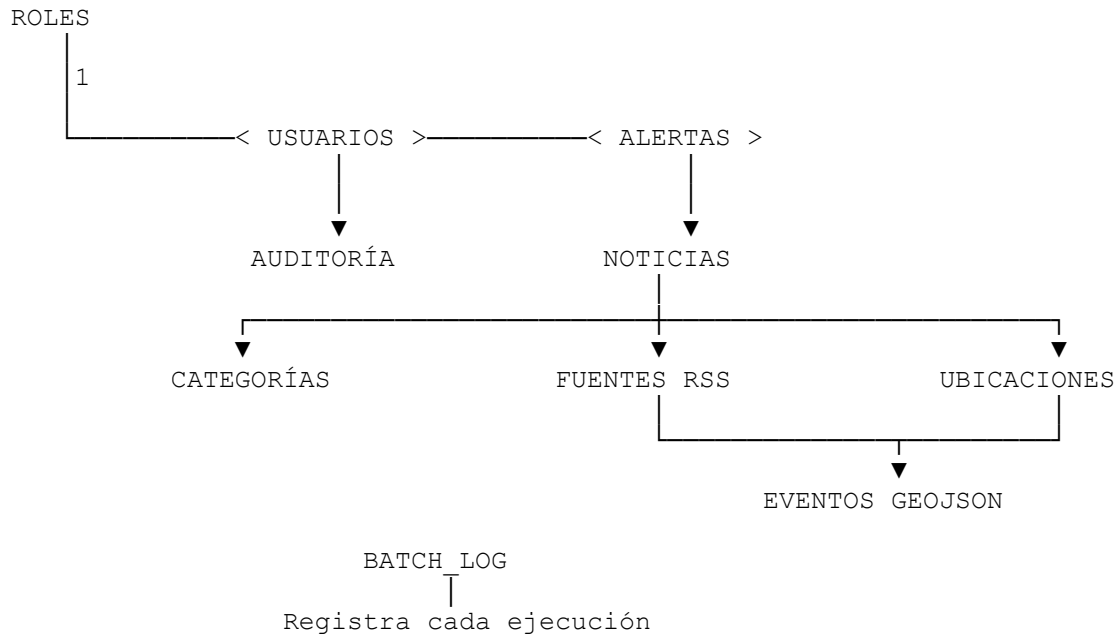
18.5 Relaciones Entre Entidades

Las principales relaciones del modelo son:

- Un **Rol** puede estar asociado a muchos **Usuarios** (1:N).
 - Un **Usuario** puede generar muchas **Alertas** (1:N).
 - Una **Fuente RSS** puede generar muchas **Noticias** (1:N).
 - Una **Categoría** puede clasificar muchas **Noticias** (1:N).
 - Una **Ubicación** puede estar asociada a muchas **Noticias** (1:N).
 - Una **Noticia** puede generar un **Evento GeoJSON** (1:1).
 - Una **Noticia** puede generar varias **Alertas** (1:N).
 - Cada ejecución del **Batch** registra un historial independiente.
-

18.6 Modelo Entidad–Relación (Descripción)

El modelo Entidad–Relación puede representarse de forma simplificada así:



18.7 Normalización

El diseño de la base de datos considera principios de normalización hasta la **Tercera Forma Normal (3FN)**.

Esto permite:

- Eliminar redundancia.
- Evitar inconsistencias.
- Facilitar actualizaciones.
- Optimizar consultas.
- Mantener integridad referencial.

18.8 Índices

Para mejorar el rendimiento del sistema se recomienda crear índices sobre los siguientes campos:

- fecha_publicación
- severidad
- categoría
- país

- ciudad
- fuente
- título
- estado

Estos índices permitirán acelerar las búsquedas y consultas realizadas desde el portal web.

18.9 Integridad Referencial

La base de datos implementará claves primarias y foráneas para garantizar la consistencia de la información.

Ejemplos:

- Noticias → Categorías.
 - Noticias → Fuentes RSS.
 - Noticias → Ubicaciones.
 - Usuarios → Roles.
 - Alertas → Usuarios.
 - Alertas → Noticias.
-

18.10 Respaldo de la Información

Con el propósito de proteger la información almacenada, el sistema implementará:

- Copias de seguridad automáticas de la base de datos.
 - Respaldo de archivos GeoJSON generados por el proceso Batch.
 - Registro histórico de noticias procesadas.
 - Recuperación ante fallos del sistema.
-

18.11 Consideraciones de Rendimiento

El diseño de la base de datos permitirá:

- Procesar miles de noticias por ejecución Batch.
- Consultar información mediante filtros.
- Visualizar mapas sin afectar el rendimiento del sistema.
- Generar estadísticas en tiempos reducidos.
- Escalar el almacenamiento conforme aumente el volumen de información.

18.12 Beneficios del Modelo de Datos

El diseño propuesto ofrece las siguientes ventajas:

- Organización estructurada de la información.
- Alta consistencia de los datos.
- Escalabilidad para incorporar nuevas entidades.
- Compatibilidad con sistemas geográficos.
- Integración sencilla con la API REST.
- Facilidad para generar reportes y estadísticas.
- Optimización del procesamiento Batch.
- Soporte para el almacenamiento de información geoespacial.

18.13 Conclusión

El diseño de la base de datos proporciona una estructura robusta para soportar todas las funcionalidades de la plataforma **Alert!**. La organización de las entidades y sus relaciones facilita el almacenamiento, procesamiento y consulta de noticias, así como la gestión de usuarios, la generación de archivos GeoJSON y la visualización de eventos georreferenciados. Este modelo constituye la base para garantizar la integridad, eficiencia y escalabilidad del sistema, permitiendo su evolución conforme aumenten las necesidades del proyecto.

Como complemento a este capítulo, es recomendable incluir los siguientes artefactos técnicos:

1. **Diagrama Entidad–Relación (ERD)** con todas las tablas y relaciones.
2. **Modelo Relacional** con claves primarias (PK) y foráneas (FK).
3. **Diccionario de Datos**, detallando nombre del campo, tipo de dato, longitud, restricciones y descripción.
4. **Script SQL** para la creación de la base de datos y sus tablas en MySQL.

Estos elementos fortalecerán la documentación técnica y facilitarán la implementación y mantenimiento de la plataforma **Alert!**.

Con gusto. A continuación se presenta el capítulo **Diagramas UML** para el proyecto **Alert!**. Este documento se fundamenta en los requerimientos funcionales, los casos de uso y la arquitectura del sistema definidos para la plataforma, integrando el proceso **Batch**, la generación de **GeoJSON**, la georreferenciación y la visualización cartográfica.

19. Diagramas UML

19.1 Introducción

El Lenguaje Unificado de Modelado (UML) es un estándar ampliamente utilizado para representar la estructura y el comportamiento de los sistemas de software. Mediante diagramas UML es posible describir de forma gráfica los componentes, las relaciones y las interacciones que conforman la plataforma **Alert!**, facilitando la comprensión del diseño, la implementación y el mantenimiento del sistema.

Los diagramas presentados en este capítulo reflejan la arquitectura modular de la aplicación, el flujo de procesamiento de noticias, la interacción entre los usuarios y el sistema, así como la organización de sus componentes principales.

19.2 Objetivos de los Diagramas UML

Los diagramas UML tienen como propósito:

- Representar gráficamente la arquitectura del sistema.
 - Identificar los actores que interactúan con la plataforma.
 - Describir el comportamiento de los procesos Batch.
 - Modelar la estructura de clases y componentes.
 - Facilitar la comunicación entre los desarrolladores.
 - Servir como guía para la implementación del software.
 - Documentar el diseño del proyecto.
-

19.3 Diagrama de Casos de Uso

El diagrama de casos de uso representa las funcionalidades principales ofrecidas por la plataforma y la interacción de los actores con dichas funcionalidades.

Actores

- Usuario Visitante
- Usuario Registrado
- Analista
- Administrador
- Sistema Batch

Casos de uso principales

- Consultar noticias.

- Buscar noticias.
- Recibir alertas.
- Administrar usuarios.
- Administrar categorías.
- Administrar fuentes RSS.
- Ejecutar proceso Batch.
- Clasificar noticias.
- Georreferenciar eventos.
- Generar archivo GeoJSON.
- Visualizar mapa de alertas.
- Consultar estadísticas.
- Generar reportes.

Representación simplificada

```

+-----+
|      Alert!      |
+-----+

```

```

Usuario -----> Consultar Noticias
Usuario -----> Buscar Noticias
Usuario -----> Ver Mapa
Usuario -----> Consultar Estadísticas

```

```

Usuario Registrado --> Recibir Alertas

```

```

Analista -----> Validar Clasificación
Analista -----> Consultar Eventos

```

```

Administrador -----> Gestionar Usuarios
Administrador -----> Gestionar Categorías
Administrador -----> Gestionar RSS

```

```

Sistema Batch -----> Capturar Noticias
Sistema Batch -----> Clasificar Noticias
Sistema Batch -----> Georreferenciar
Sistema Batch -----> Generar GeoJSON

```

19.4 Diagrama de Clases

El diagrama de clases representa las entidades principales del sistema y las relaciones existentes entre ellas.

Clases principales

- Usuario
- Rol
- Noticia
- Categoría
- FuenteRSS

- Ubicación
- EventoGeoJSON
- Alerta
- BatchProcessor
- Reporte

Relaciones

- Usuario pertenece a un Rol.
- Noticia pertenece a una Categoría.
- Noticia proviene de una Fuente RSS.
- Noticia tiene una Ubicación.
- EventoGeoJSON representa una Noticia.
- BatchProcessor genera Noticias y EventosGeoJSON.

Representación simplificada

ROL

idRol
nombre

1
|
|
N

USUARIO

idUsuario
nombre
correo
password

NOTICIA

id
titulo
contenido
fecha
severidad

|1
|
N

CATEGORIA

FUENTE RSS

UBICACION

EVENTO GEOJSON

ALERTA

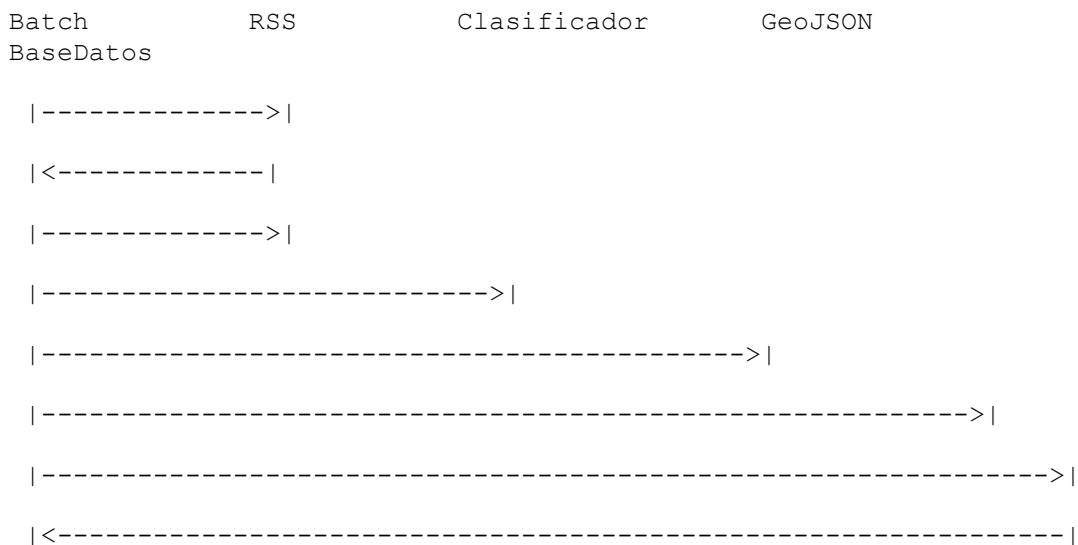
19.5 Diagrama de Secuencia

El diagrama de secuencia describe el proceso Batch encargado de recopilar y procesar las noticias.

Flujo del proceso

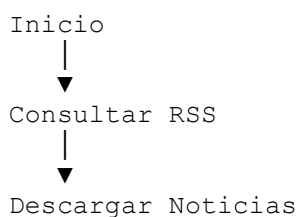
1. El Batch inicia.
2. Consulta las fuentes RSS.
3. Descarga las noticias.
4. Elimina duplicados.
5. Clasifica las noticias.
6. Identifica ubicaciones.
7. Obtiene coordenadas.
8. Genera el archivo GeoJSON.
9. Guarda la información en la base de datos.
10. Publica la información en la aplicación.

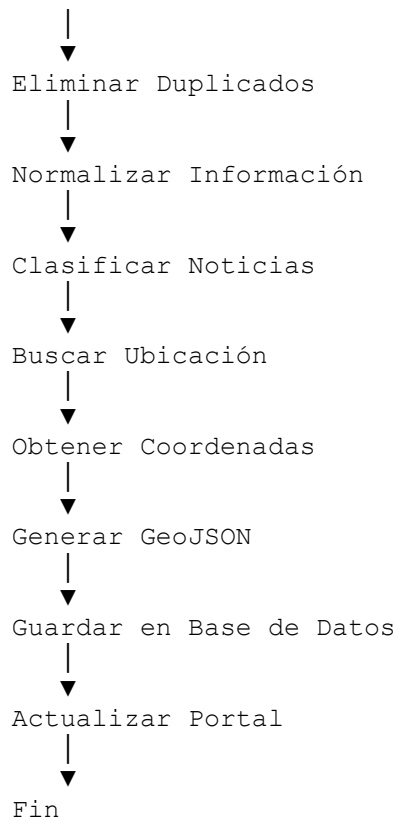
Representación



19.6 Diagrama de Actividades

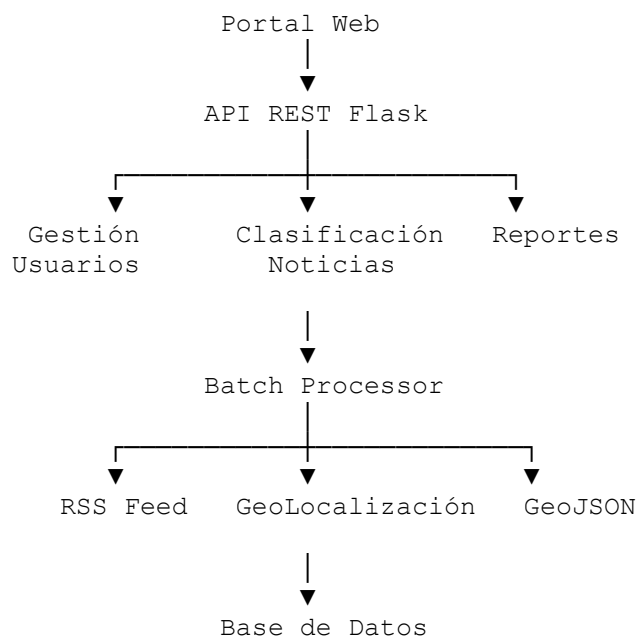
Este diagrama representa el flujo de trabajo del procesamiento de noticias.





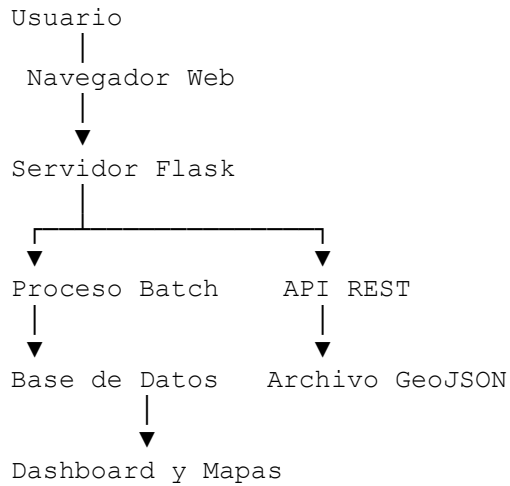
19.7 Diagrama de Componentes

El sistema está conformado por los siguientes componentes:



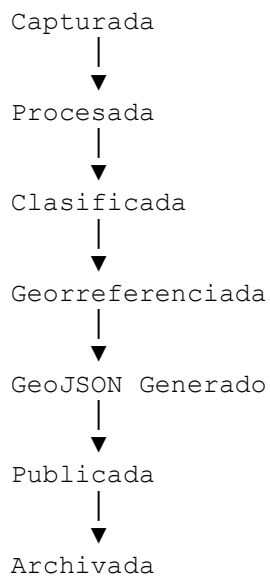
19.8 Diagrama de Despliegue

Describe la infraestructura física donde opera la plataforma.



19.9 Diagrama de Estados

El ciclo de vida de una noticia puede representarse mediante los siguientes estados:



19.10 Diagrama de Paquetes

La organización lógica del proyecto puede estructurarse en los siguientes paquetes:

Alert!

|

- autenticacion
- usuarios
- noticias
- batch
- clasificacion
- geolocalizacion
- geojson
- mapas
- reportes
- api
- base_datos
- utilidades

19.11 Diagrama de Comunicación

Representa el intercambio de información entre los componentes.

```
Usuario
|
Portal Web
|
Flask
|
API REST
|
Base de Datos
|
GeoJSON
|
Leaflet
```

19.12 Beneficios del Modelado UML

El uso de UML en el proyecto **Alert!** proporciona las siguientes ventajas:

- Facilita la comprensión del sistema.
 - Mejora la comunicación entre los integrantes del equipo.
 - Reduce errores durante el desarrollo.
 - Permite documentar la arquitectura.
 - Favorece el mantenimiento del software.
 - Facilita la incorporación de nuevos módulos.
 - Sirve como referencia para las pruebas y la evolución del sistema.
-

19.13 Conclusión

Los diagramas UML desarrollados para la plataforma **Alert!** describen la estructura y el comportamiento del sistema desde diferentes perspectivas, permitiendo representar las

funcionalidades, los componentes, las entidades y el flujo de procesamiento de noticias. Estos modelos constituyen una guía para la implementación del software y fortalecen la documentación técnica del proyecto al proporcionar una visión integral de su arquitectura y funcionamiento.

Para un proyecto de grado o del **SENA**, este capítulo debe complementarse con **diagramas UML elaborados en una herramienta de modelado** (como StarUML, Visual Paradigm, Lucidchart, Draw.io o Microsoft Visio). Se recomienda incluir al menos los siguientes diagramas con calidad profesional:

1. **Diagrama de Casos de Uso** (5 actores y 12–15 casos de uso).
2. **Diagrama de Clases** (10–15 clases con atributos, métodos y relaciones).
3. **Diagrama de Secuencia** del proceso Batch.
4. **Diagrama de Actividades** del procesamiento de noticias.
5. **Diagrama de Componentes** de la arquitectura.
6. **Diagrama de Despliegue** (servidor, base de datos, API y cliente web).
7. **Diagrama de Estados** para el ciclo de vida de una noticia.
8. **Diagrama de Paquetes** con la organización del código fuente.

Estos diagramas aportarán mayor solidez técnica a la documentación del proyecto **Alert!**.

Con gusto. A continuación se presenta el capítulo **Diseño de Interfaces** para el proyecto **Alert!**. Este documento se fundamenta en los requerimientos funcionales, los casos de uso y la arquitectura del sistema definidos para la plataforma, considerando las funcionalidades del prototipo, como la consulta de noticias, el procesamiento Batch, la visualización cartográfica y la administración del sistema.

20. Diseño de Interfaces

20.1 Introducción

El diseño de interfaces de usuario (UI) constituye un elemento fundamental para garantizar una interacción eficiente entre los usuarios y la plataforma **Alert!**. Una interfaz intuitiva y organizada facilita el acceso a las funcionalidades del sistema, mejora la experiencia del usuario y optimiza la consulta y el análisis de la información georreferenciada.

El diseño de la aplicación se basa en principios de simplicidad, accesibilidad, consistencia visual y adaptabilidad, permitiendo que los diferentes perfiles de usuario interactúen de forma clara con las funcionalidades de monitoreo, clasificación y visualización de noticias.

20.2 Objetivos del Diseño de Interfaces

El diseño de interfaces tiene como objetivos:

- Proporcionar una navegación intuitiva.
 - Facilitar la consulta de noticias.
 - Permitir la visualización geográfica de incidentes.
 - Optimizar el acceso a estadísticas y reportes.
 - Mejorar la experiencia del usuario.
 - Garantizar una interfaz adaptable a diferentes dispositivos.
-

20.3 Principios de Diseño

La interfaz de Alert! se desarrollará siguiendo los siguientes principios:

Simplicidad

Las pantallas mostrarán únicamente la información necesaria para cada proceso.

Consistencia

Los elementos gráficos mantendrán colores, tipografías, iconografía y distribución homogénea en toda la aplicación.

Accesibilidad

La plataforma procurará cumplir con buenas prácticas de accesibilidad, utilizando contrastes adecuados, textos legibles y controles de fácil interacción.

Retroalimentación

El sistema informará al usuario sobre el estado de las operaciones mediante mensajes de confirmación, advertencias y notificaciones.

Diseño Responsivo

La interfaz será compatible con computadores, tabletas y dispositivos móviles.

20.4 Estructura General de la Interfaz

La aplicación estará organizada en los siguientes módulos:

- Página de inicio.
- Inicio de sesión.
- Panel principal (Dashboard).
- Consulta de noticias.
- Mapa interactivo.

- Estadísticas.
- Reportes.
- Administración.
- Configuración.
- Perfil de usuario.

Cada módulo estará disponible de acuerdo con el rol del usuario.

20.5 Pantalla de Inicio

La pantalla principal será el punto de acceso a la plataforma.

Elementos principales

- Logotipo del proyecto Alert!
- Menú principal.
- Buscador de noticias.
- Acceso al mapa.
- Noticias destacadas.
- Acceso al inicio de sesión.

Propósito: ofrecer una visión general del sistema y facilitar el acceso a las funciones más utilizadas.

20.6 Interfaz de Inicio de Sesión

Permitirá la autenticación de los usuarios registrados.

Componentes

- Campo de correo electrónico o usuario.
- Campo de contraseña.
- Botón "Iniciar sesión".
- Opción de recuperación de contraseña.
- Enlace para registro de nuevos usuarios.

El acceso estará protegido mediante mecanismos de autenticación segura.

20.7 Dashboard Principal

El Dashboard centralizará la información más relevante del sistema.

Información mostrada

- Total de noticias procesadas.
- Número de alertas activas.
- Eventos críticos.
- Distribución por categorías.
- Noticias recientes.
- Estado del proceso Batch.
- Indicadores estadísticos.

El Dashboard proporcionará una visión rápida del comportamiento de los eventos monitoreados.

20.8 Interfaz de Consulta de Noticias

Permitirá explorar las noticias almacenadas.

Funcionalidades

- Búsqueda por palabras clave.
 - Filtros por fecha, país, ciudad y categoría.
 - Ordenamiento por severidad.
 - Visualización del detalle de cada noticia.
 - Acceso a la ubicación geográfica del evento.
-

20.9 Interfaz del Mapa de Alertas

Esta pantalla representará gráficamente los eventos georreferenciados.

Componentes

- Mapa interactivo.
- Marcadores geográficos.
- Código de colores según la severidad.
- Ventanas emergentes con información del evento.
- Controles de zoom y desplazamiento.
- Filtros geográficos.

El mapa utilizará la información contenida en los archivos GeoJSON generados por el proceso Batch.

20.10 Interfaz de Estadísticas

Permitirá analizar la información procesada mediante indicadores gráficos.

Elementos

- Gráficos de barras.
- Gráficos circulares.
- Series temporales.
- Indicadores por categoría.
- Distribución geográfica.
- Tendencias históricas.

Esta interfaz facilitará el análisis del comportamiento de los incidentes registrados.

20.11 Interfaz de Reportes

Permitirá generar documentos con la información almacenada.

Opciones disponibles

- Reportes diarios.
 - Reportes mensuales.
 - Reportes por categoría.
 - Reportes por país.
 - Reportes por severidad.
 - Exportación en PDF, CSV y GeoJSON.
-

20.12 Interfaz de Administración

Disponible únicamente para administradores.

Funciones

- Gestión de usuarios.
- Administración de categorías.

- Configuración de fuentes RSS.
 - Configuración del sistema.
 - Consulta de auditoría.
 - Programación del proceso Batch.
-

20.13 Perfil del Usuario

Permitirá administrar la información personal del usuario.

Opciones

- Actualizar datos personales.
 - Cambiar contraseña.
 - Configurar preferencias.
 - Gestionar alertas personalizadas.
 - Consultar historial de actividad.
-

20.14 Navegación

La navegación del sistema estará basada en un menú principal permanente con acceso a:

- Inicio.
- Noticias.
- Mapa.
- Estadísticas.
- Reportes.
- Administración (según permisos).
- Perfil.

Además, se utilizarán botones de acción rápida para las operaciones más frecuentes.

20.15 Diseño Visual

La identidad gráfica del sistema estará orientada a transmitir claridad y facilitar la interpretación de la información.

Colores sugeridos

- Azul: navegación y acciones principales.
- Verde: eventos de baja severidad.

- Amarillo: eventos moderados.
- Naranja: eventos de alta prioridad.
- Rojo: eventos críticos.

Tipografía

Se emplearán fuentes modernas y de fácil lectura, con jerarquías claras para títulos, subtítulos y contenido.

Iconografía

Se utilizarán iconos representativos para:

- Noticias.
- Ubicación.
- Alertas.
- Estadísticas.
- Usuarios.
- Configuración.
- Reportes.

20.16 Prototipos de Interfaces

Se recomienda desarrollar prototipos de alta fidelidad para las siguientes pantallas:

1. Pantalla de Inicio.
2. Inicio de Sesión.
3. Dashboard Principal.
4. Consulta de Noticias.
5. Mapa de Alertas.
6. Estadísticas.
7. Reportes.
8. Administración.
9. Perfil del Usuario.

Estos prototipos pueden elaborarse con herramientas como Figma, Adobe XD o Pencil Project antes de su implementación.

20.17 Consideraciones de Usabilidad

El diseño de interfaces deberá cumplir con los siguientes criterios:

- Navegación intuitiva.

- Tiempo mínimo para localizar información.
 - Consistencia en la distribución de elementos.
 - Mensajes claros para el usuario.
 - Compatibilidad con distintos navegadores.
 - Adaptación a diferentes tamaños de pantalla.
 - Validación de formularios en tiempo real.
-

20.18 Conclusión

El diseño de interfaces propuesto para **Alert!** proporciona una estructura organizada, intuitiva y orientada al usuario, permitiendo acceder de forma eficiente a las funcionalidades de monitoreo, clasificación, georreferenciación y visualización de noticias. La distribución modular de las pantallas, junto con el uso de componentes visuales consistentes y un diseño responsivo, favorecerá la usabilidad y mejorará la experiencia de los diferentes perfiles de usuario que interactúan con la plataforma.

Recomendación: Para fortalecer este capítulo, es conveniente incluir **mockups o prototipos gráficos** de las principales pantallas (Inicio, Login, Dashboard, Mapa de Alertas, Noticias, Estadísticas y Administración). Estos diseños pueden elaborarse en herramientas como **Figma** y servirán como referencia directa para la implementación del frontend de la aplicación.

Con gusto. A continuación se presenta el capítulo **Cronograma de Desarrollo (Diagrama de Gantt)** para el proyecto **Alert!**. Este cronograma está alineado con la metodología Scrum planteada anteriormente y distribuye el desarrollo del sistema en fases que abarcan desde el análisis de requisitos hasta las pruebas y la documentación final.

21. Cronograma de Desarrollo (Diagrama de Gantt)

21.1 Introducción

El cronograma de desarrollo constituye una herramienta de planificación que permite organizar las actividades del proyecto, establecer tiempos de ejecución, asignar responsabilidades y realizar seguimiento al avance del desarrollo de la plataforma **Alert!**.

Para este proyecto se propone un cronograma de **24 semanas (6 meses)**, organizado en fases consecutivas e iterativas, siguiendo la metodología ágil **Scrum**. Cada fase contempla actividades específicas relacionadas con el análisis, diseño, implementación, pruebas y documentación del sistema.

21.2 Objetivo del Cronograma

El cronograma tiene como propósito:

- Planificar las actividades del proyecto.
- Controlar el avance del desarrollo.
- Organizar los tiempos de cada fase.
- Facilitar el seguimiento del proyecto.
- Identificar dependencias entre actividades.
- Garantizar el cumplimiento de los objetivos establecidos.

21.3 Fases del Proyecto

El desarrollo de la plataforma **Alert!** comprende las siguientes fases:

1. Planeación del proyecto.
2. Levantamiento de requisitos.
3. Diseño del sistema.
4. Desarrollo del procesamiento Batch.
5. Desarrollo del portal web.
6. Desarrollo del módulo de georreferenciación.
7. Generación del GeoJSON.
8. Integración del sistema.
9. Pruebas.
10. Documentación.
11. Despliegue.

21.4 Cronograma General

[illegible]

Actividad	S	S	S	S	S	S	S	S	S	S	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S2	S2	S2	S2	S2
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	
Diseño de interfaces				•	•																				
Desarrollo del Batch					•	•	•	•																	
Captura de noticias RSS				•	•	•																			
Clasificación automática					•	•	•																		
Georreferenciación							•	•	•																
Generación GeoJSON							•	•																	
Desarrollo portal web							•	•	•	•	•	•													
API REST							•	•	•																
Dashboard							•	•	•	•	•	•													
Mapa interactivo							•	•	•	•	•	•													
Reportes										•	•	•	•												
Integración del sistema										•	•	•	•												
Pruebas unitarias											•	•	•	•											
Pruebas de integración												•	•	•	•										
Optimización													•	•	•										
Documentación técnica														•	•	•	•	•							
Manual de usuario															•	•	•								
Despliegue																	•	•	•						
Validación final																		•	•	•					

Actividad	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S1	S2	S2	S2	S2	S2
	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	
Entrega del proyecto																									

Leyenda: • = Semana en la que se desarrolla la actividad.

21.5 Distribución por Etapas

Etapas 1. Análisis (Semanas 1 a 4)

- Planeación del proyecto.
- Levantamiento de requisitos.
- Casos de uso.
- Arquitectura.
- Diseño de la base de datos.

Etapas 2. Diseño (Semanas 4 a 6)

- Diseño de interfaces.
- Diseño UML.
- Preparación del entorno de desarrollo.

Etapas 3. Desarrollo (Semanas 5 a 14)

- Desarrollo del proceso Batch.
- Captura automática de noticias.
- Clasificación automática.
- Georreferenciación.
- Generación de archivos GeoJSON.
- Desarrollo del portal web.
- Implementación de la API REST.
- Dashboard y mapas interactivos.

Etapas 4. Integración y Pruebas (Semanas 12 a 18)

- Integración de módulos.
- Pruebas unitarias.
- Pruebas de integración.

- Corrección de errores.
- Optimización del rendimiento.

Etapa 5. Documentación y Despliegue (Semanas 16 a 24)

- Elaboración de documentación técnica.
 - Manual de usuario.
 - Manual técnico.
 - Despliegue de la aplicación.
 - Validación final.
 - Entrega del proyecto.
-

21.6 Hitos del Proyecto

Hito	Semana
Finalización del análisis	4
Arquitectura aprobada	4
Base de datos implementada	6
Proceso Batch funcional	8
Generación de GeoJSON	10
Portal web operativo	12
Sistema integrado	16
Finalización de pruebas	18
Documentación completa	22
Entrega del proyecto	24

21.7 Seguimiento del Cronograma

El seguimiento del proyecto se realizará mediante reuniones periódicas de revisión (Sprint Review), en las que se verificará el cumplimiento de las actividades programadas, se identificarán riesgos y se propondrán acciones de mejora para garantizar el avance del desarrollo.

21.8 Conclusión

El cronograma de desarrollo propuesto proporciona una planificación estructurada para la construcción de la plataforma **Alert!**, distribuyendo las actividades en cinco etapas principales. La organización temporal facilita el control del avance del proyecto, la asignación eficiente de recursos y el cumplimiento de los objetivos establecidos, permitiendo desarrollar una solución robusta para el monitoreo automatizado de noticias, la generación de archivos GeoJSON y la visualización geoespacial de eventos.

Como complemento a este capítulo, es recomendable incluir un **Diagrama de Gantt gráfico** elaborado en **Microsoft Project, ProjectLibre, GanttProject o Excel**, con barras de tiempo, dependencias entre tareas e hitos. Este tipo de representación visual suele ser un requisito en proyectos del **SENA** y facilita la planificación y el seguimiento del desarrollo.

Con gusto. A continuación se presenta el capítulo **Presupuesto del Proyecto** para la formulación del proyecto de software **Alert!**. Este presupuesto es una estimación para el desarrollo de un prototipo funcional durante un período aproximado de **6 meses**, considerando costos de personal, software, infraestructura y otros recursos.

22. Presupuesto del Proyecto

22.1 Introducción

El presupuesto del proyecto constituye una estimación de los recursos financieros necesarios para desarrollar la plataforma **Alert!**, destinada al monitoreo automatizado de noticias, clasificación de eventos, georreferenciación y generación de archivos GeoJSON.

La estimación incluye los costos asociados al talento humano, equipos, infraestructura tecnológica, licencias, conectividad y gastos operativos requeridos para el desarrollo del prototipo.

22.2 Objetivo del Presupuesto

El presupuesto tiene como finalidad:

- Estimar el costo total del proyecto.
 - Identificar los recursos necesarios.
 - Facilitar la planificación financiera.
 - Evaluar la viabilidad económica del proyecto.
 - Servir como referencia para futuras implementaciones.
-

22.3 Recursos Humanos

Cargo	Cantidad	Tiempo	Valor Mensual (COP)	Subtotal (COP)
Director del Proyecto	1	6 meses	\$4.500.000	\$27.000.000
Analista de Software	1	6 meses	\$3.800.000	\$22.800.000
Desarrollador Backend	1	6 meses	\$4.000.000	\$24.000.000
Desarrollador Frontend	1	6 meses	\$3.800.000	\$22.800.000
Ingeniero de Base de Datos	1	3 meses	\$4.000.000	\$12.000.000
Ingeniero QA (Pruebas)	1	2 meses	\$3.500.000	\$7.000.000
Diseñador UX/UI	1	2 meses	\$3.500.000	\$7.000.000

Total Recursos Humanos

\$122.600.000 COP

22.4 Hardware

Recurso	Cantidad	Valor Unitario	Subtotal
Computador de Desarrollo	2	\$4.500.000	\$9.000.000
Monitor adicional	2	\$800.000	\$1.600.000
Disco SSD Externo	2	\$450.000	\$900.000
UPS	1	\$800.000	\$800.000

Total Hardware

\$12.300.000 COP

22.5 Software

Recurso	Licencia	Valor
Visual Studio Code	Gratuito	\$0
Python	Gratuito	\$0

Recurso	Licencia	Valor
Flask	Gratuito	\$0
MySQL Community	Gratuito	\$0
Git	Gratuito	\$0
GitHub	Gratuito	\$0
Figma (Plan Profesional)	6 meses	\$1.000.000
Microsoft Project (opcional)	6 meses	\$1.500.000

Total Software

\$2.500.000 COP

22.6 Infraestructura

Concepto	Valor
Hosting VPS (6 meses)	\$1.800.000
Dominio web (.com)	\$80.000
Certificado SSL	\$250.000
Servicio de respaldo en la nube	\$500.000

Total Infraestructura

\$2.630.000 COP

22.7 Conectividad

Concepto	Valor
Servicio de Internet (6 meses)	\$900.000
Servicios de energía eléctrica	\$1.200.000

Total Conectividad

\$2.100.000 COP

22.8 Gastos Administrativos

Concepto	Valor
Papelería y útiles	\$300.000
Impresiones	\$200.000
Reuniones y logística	\$500.000
Transporte	\$600.000

Total Gastos Administrativos

\$1.600.000 COP

22.9 Contingencias

Se destina un fondo equivalente al **10 %** del presupuesto base para cubrir imprevistos, ajustes técnicos o necesidades adicionales durante el desarrollo.

Concepto	Valor
Fondo de contingencia	\$14.213.000

22.10 Resumen General del Presupuesto

Concepto	Valor (COP)
Recursos Humanos	\$122.600.000
Hardware	\$12.300.000
Software	\$2.500.000
Infraestructura	\$2.630.000
Conectividad	\$2.100.000
Gastos Administrativos	\$1.600.000
Contingencias	\$14.213.000

Costo Total Estimado del Proyecto

\$157.943.000 COP

22.11 Presupuesto para un Proyecto Académico (SENA)

En caso de desarrollar **Alert!** como proyecto de formación o trabajo de grado, utilizando recursos propios y herramientas de software libre, el presupuesto se reduce significativamente.

Concepto	Valor (COP)
Desarrollo realizado por aprendices	\$0
Software de código abierto	\$0
GitHub	\$0
Python	\$0
Flask	\$0
MySQL Community	\$0
Visual Studio Code	\$0
Hosting gratuito o institucional	\$0
Dominio (opcional)	\$80.000
Internet	\$900.000
Energía	\$1.200.000
Papelería	\$300.000

Total estimado para proyecto académico

\$2.480.000 COP

22.12 Fuentes de Financiación

El proyecto puede financiarse mediante:

- Recursos propios del desarrollador.
 - Instituciones de educación superior.
 - Convocatorias de investigación.
 - Fondos de innovación y emprendimiento.
 - Entidades gubernamentales.
 - Alianzas con empresas del sector tecnológico.
-

22.13 Viabilidad Económica

El desarrollo de la plataforma **Alert!** resulta económicamente viable, especialmente cuando se aprovechan tecnologías de código abierto como Python, Flask, MySQL y Git, las cuales reducen significativamente los costos de licenciamiento. Para un entorno empresarial, la mayor inversión corresponde al talento humano, mientras que para un proyecto académico los costos se concentran principalmente en infraestructura básica y conectividad.

22.14 Conclusión

El presupuesto estimado demuestra que la implementación de **Alert!** puede adaptarse a diferentes escenarios. En un contexto profesional, el costo aproximado es de **\$157.943.000 COP**, mientras que en un entorno académico, haciendo uso de herramientas gratuitas y recursos institucionales, el costo puede reducirse a aproximadamente **\$2.480.000 COP**. Esta flexibilidad facilita la ejecución del proyecto y permite su evolución hacia una solución escalable y sostenible.

Recomendación: Si este documento será presentado como **proyecto de grado del SENA**, es conveniente agregar un **presupuesto detallado en Excel**, con fórmulas automáticas, porcentajes de participación por rubro y gráficos (pastel y barras), ya que este formato suele ser solicitado para el análisis financiero del proyecto.

Con gusto. A continuación se presenta el capítulo **Gestión de Riesgos** para el proyecto **Alert!**. El documento está orientado a una formulación de proyecto de software bajo buenas prácticas de gestión de proyectos y considera los riesgos asociados al desarrollo de una plataforma de monitoreo automatizado de noticias, procesamiento Batch, georreferenciación y generación de archivos GeoJSON.

23. Gestión de Riesgos

23.1 Introducción

La gestión de riesgos constituye un proceso fundamental dentro del desarrollo de proyectos de software, ya que permite identificar, analizar, evaluar y establecer

estrategias para prevenir o mitigar eventos que puedan afectar el cumplimiento de los objetivos del proyecto.

En la plataforma **Alert!**, la gestión de riesgos adquiere especial importancia debido a la integración de múltiples tecnologías, el procesamiento automatizado de noticias mediante procesos Batch, la dependencia de fuentes externas de información y la generación de datos geoespaciales. Una adecuada administración de los riesgos contribuye a garantizar la continuidad del proyecto, la calidad del software y el cumplimiento de los tiempos y costos establecidos.

23.2 Objetivos de la Gestión de Riesgos

La gestión de riesgos tiene como objetivos:

- Identificar los riesgos potenciales del proyecto.
 - Analizar la probabilidad de ocurrencia y su impacto.
 - Definir estrategias de prevención y mitigación.
 - Reducir la incertidumbre durante el desarrollo.
 - Proteger la calidad del producto final.
 - Favorecer la toma de decisiones oportunas.
-

23.3 Proceso de Gestión de Riesgos

La gestión de riesgos en el proyecto **Alert!** comprende las siguientes etapas:

1. Identificación de riesgos.
2. Análisis de probabilidad e impacto.
3. Evaluación y priorización.
4. Definición de estrategias de respuesta.
5. Monitoreo y seguimiento.
6. Actualización del registro de riesgos.

Este proceso será continuo durante todo el ciclo de vida del proyecto.

23.4 Identificación de Riesgos

Los principales riesgos identificados son:

Riesgos Técnicos

- Fallas en el proceso Batch.
 - Cambios en el formato de las fuentes RSS.
 - Errores en la clasificación automática de noticias.
 - Problemas de georreferenciación.
 - Pérdida de información.
 - Baja calidad de los datos obtenidos.
 - Incompatibilidad entre bibliotecas de software.
-

Riesgos Tecnológicos

- Fallas del servidor.
 - Indisponibilidad del servicio de alojamiento.
 - Fallos en la conexión a Internet.
 - Problemas con la base de datos.
 - Obsolescencia de herramientas o dependencias.
-

Riesgos del Proyecto

- Retrasos en el cronograma.
 - Cambios en los requisitos.
 - Limitaciones presupuestales.
 - Falta de disponibilidad de recursos.
 - Retrasos en las actividades de desarrollo.
-

Riesgos Operacionales

- Errores humanos durante la configuración.
 - Eliminación accidental de información.
 - Configuración incorrecta del proceso Batch.
 - Fallas durante la implementación.
-

Riesgos de Seguridad

- Acceso no autorizado.
 - Robo de credenciales.
 - Ataques informáticos.
 - Modificación no autorizada de la información.
 - Pérdida de integridad de la base de datos.
-

23.5 Análisis de Riesgos

Cada riesgo será evaluado considerando dos criterios:

- **Probabilidad de ocurrencia** (Baja, Media o Alta).
- **Impacto sobre el proyecto** (Bajo, Medio o Alto).

La combinación de ambos criterios permitirá establecer la prioridad de atención.

23.6 Matriz de Riesgos

Código	Riesgo	Probabilidad	Impacto	Nivel	Estrategia
R-01	Caída del servidor	Media	Alto	Alto	Infraestructura redundante y monitoreo
R-02	Fallo del proceso Batch	Media	Alto	Alto	Registro de errores y reintentos automáticos
R-03	Cambios en fuentes RSS	Alta	Medio	Alto	Validación periódica y actualización de conectores
R-04	Errores de georreferenciación	Media	Medio	Medio	Validación manual de casos críticos
R-05	Pérdida de datos	Baja	Alto	Alto	Copias de seguridad automáticas
R-06	Acceso no autorizado	Media	Alto	Alto	Autenticación segura y control de acceso
R-07	Retrasos en el cronograma	Media	Medio	Medio	Seguimiento semanal del proyecto
R-08	Cambios en los requisitos	Media	Medio	Medio	Gestión formal de cambios
R-09	Bajo rendimiento del sistema	Baja	Alto	Medio	Optimización y pruebas de rendimiento
R-10	Dependencias de software incompatibles	Baja	Medio	Bajo	Gestión de versiones y pruebas continuas

23.7 Estrategias de Respuesta

Para cada riesgo identificado se definirá una estrategia de respuesta:

Evitar

Modificar el enfoque del proyecto para eliminar la causa del riesgo.

Ejemplo:

- Utilizar bibliotecas estables y ampliamente soportadas.
-

Mitigar

Reducir la probabilidad o el impacto mediante acciones preventivas.

Ejemplo:

- Implementar pruebas automáticas.
 - Monitorear el proceso Batch.
 - Validar periódicamente las fuentes RSS.
-

Transferir

Delegar parte del riesgo mediante servicios externos o contratos.

Ejemplo:

- Utilizar proveedores de hosting con acuerdos de nivel de servicio (SLA).
-

Aceptar

Aceptar el riesgo cuando su impacto sea bajo o el costo de mitigarlo sea superior al beneficio esperado.

Ejemplo:

- Ajustes menores derivados de cambios no críticos en el formato de las noticias.
-

23.8 Plan de Mitigación

Las principales acciones de mitigación incluyen:

- Realizar copias de seguridad diarias de la base de datos.
- Implementar registro (logs) de todas las ejecuciones del proceso Batch.
- Validar automáticamente la información descargada.
- Monitorear el consumo de recursos del servidor.
- Utilizar control de versiones con Git.
- Documentar todos los cambios realizados.
- Ejecutar pruebas periódicas de integración.
- Capacitar a los usuarios administradores.

23.9 Monitoreo y Seguimiento

El seguimiento de los riesgos se realizará durante todo el desarrollo del proyecto mediante:

- Reuniones de seguimiento al final de cada Sprint.
- Actualización del registro de riesgos.
- Revisión de incidentes.
- Análisis de métricas del sistema.
- Seguimiento al cumplimiento del cronograma.

Los riesgos identificados serán reevaluados periódicamente para determinar si han cambiado su probabilidad o impacto.

23.10 Roles y Responsabilidades

Rol	Responsabilidades
Director del Proyecto	Supervisar la gestión integral de riesgos
Analista de Software	Identificar y documentar riesgos funcionales
Desarrolladores	Implementar acciones preventivas y correctivas
Ingeniero QA	Verificar la calidad y detectar riesgos técnicos
Administrador del Sistema	Monitorear infraestructura y seguridad

23.11 Herramientas de Gestión

Durante el proyecto podrán utilizarse las siguientes herramientas:

- Git y GitHub para el control de versiones.
 - Trello o Jira para el seguimiento de tareas.
 - Visual Studio Code para el desarrollo.
 - MySQL Workbench para la administración de la base de datos.
 - Herramientas de monitoreo del servidor y análisis de registros (logs).
-

23.12 Beneficios de la Gestión de Riesgos

La aplicación de un proceso de gestión de riesgos aporta las siguientes ventajas:

- Disminuye la probabilidad de fallos críticos.
 - Reduce retrasos en el cronograma.
 - Mejora la calidad del software.
 - Incrementa la disponibilidad del sistema.
 - Protege la información procesada.
 - Facilita la toma de decisiones.
 - Favorece el cumplimiento de los objetivos del proyecto.
-

23.13 Conclusión

La gestión de riesgos representa un componente esencial para el éxito del proyecto **Alert!**, ya que permite anticipar posibles amenazas técnicas, operativas, tecnológicas y de seguridad. La identificación temprana de los riesgos, junto con la implementación de estrategias de prevención, mitigación y seguimiento continuo, contribuirá a desarrollar una plataforma confiable, segura y escalable para el monitoreo automatizado de noticias y la generación de información geoespacial.

Recomendación: Para fortalecer este capítulo en un proyecto de grado o del **SENA**, es conveniente incluir un **Registro de Riesgos (Risk Register)** en una hoja de cálculo, con campos como: código del riesgo, descripción, causa, probabilidad, impacto, nivel de riesgo, responsable, estrategia de respuesta, estado y fecha de revisión. Esto facilitará el seguimiento y la actualización de los riesgos durante todo el ciclo de vida del proyecto.

Con gusto. A continuación se presenta el capítulo **Plan de Pruebas** para el proyecto **Alert!**. Este documento está alineado con la arquitectura, los requerimientos funcionales y no funcionales, y sigue las recomendaciones de la norma **ISO/IEC/IEEE 29119** para la gestión y ejecución de pruebas de software.

24. Plan de Pruebas

24.1 Introducción

El Plan de Pruebas define la estrategia para verificar y validar el correcto funcionamiento de la plataforma **Alert!**, garantizando que el sistema cumpla con los requisitos funcionales y no funcionales establecidos durante la formulación del proyecto.

Las pruebas permitirán identificar errores, validar la calidad del software y asegurar que los módulos desarrollados funcionen de forma integrada antes de su puesta en producción.

24.2 Objetivos del Plan de Pruebas

El plan de pruebas tiene como objetivos:

- Verificar el cumplimiento de los requisitos funcionales.
 - Validar el desempeño del sistema.
 - Detectar errores antes de la implementación.
 - Evaluar la integración entre los módulos.
 - Garantizar la estabilidad y confiabilidad de la plataforma.
 - Comprobar la correcta generación de archivos GeoJSON.
-

24.3 Alcance

Las pruebas cubrirán todos los módulos de la plataforma:

- Inicio de sesión.
 - Gestión de usuarios.
 - Captura de noticias RSS.
 - Proceso Batch.
 - Clasificación automática.
 - Georreferenciación.
 - Generación del archivo GeoJSON.
 - Base de datos.
 - API REST.
 - Dashboard.
 - Mapa interactivo.
 - Reportes.
 - Estadísticas.
-

24.4 Estrategia de Pruebas

La estrategia contempla diferentes niveles de prueba:

Pruebas Unitarias

Validan el funcionamiento individual de cada función, clase o módulo desarrollado.

Ejemplos:

- Lectura de una fuente RSS.
 - Clasificación de una noticia.
 - Conversión de coordenadas.
 - Generación de un objeto GeoJSON.
-

Pruebas de Integración

Verifican la interacción entre los diferentes componentes.

Ejemplos:

- Integración entre el Batch y la base de datos.
 - Integración entre la API REST y el portal web.
 - Integración entre la georreferenciación y el mapa.
-

Pruebas Funcionales

Comprueban que las funcionalidades implementadas cumplan con los requisitos definidos.

Ejemplos:

- Registro e inicio de sesión de usuarios.
 - Consulta de noticias.
 - Filtrado por categoría.
 - Visualización del mapa.
-

Pruebas de Sistema

Evalúan el comportamiento de la plataforma completa en un entorno similar al de producción.

Se verificará:

- Flujo completo del procesamiento Batch.
- Generación de GeoJSON.
- Consulta desde la interfaz web.
- Generación de reportes.

Pruebas de Aceptación

Serán realizadas por los usuarios finales para confirmar que el sistema satisface sus necesidades y expectativas.

24.5 Ambiente de Pruebas

Las pruebas se ejecutarán en un entorno controlado con la siguiente configuración:

Componente	Configuración
Sistema Operativo	Windows 10 / Ubuntu Linux
Lenguaje	Python 3.x
Framework	Flask
Base de datos	MySQL
Navegadores	Chrome, Edge y Firefox
Servidor web	Flask Development Server o Gunicorn
Herramientas de prueba Pytest, Postman y MySQL Workbench	

24.6 Casos de Prueba

Caso de Prueba CP-01

Nombre: Inicio de sesión

Objetivo: Validar el acceso al sistema.

Entradas:

- Usuario válido.
- Contraseña válida.

Resultado esperado:

El sistema autentica al usuario y muestra el Dashboard.

Caso de Prueba CP-02

Nombre: Captura de noticias RSS

Objetivo: Verificar la descarga de noticias.

Resultado esperado:

Las noticias se almacenan correctamente en la base de datos.

Caso de Prueba CP-03

Nombre: Clasificación automática

Objetivo: Validar la asignación de categorías y niveles de severidad.

Resultado esperado:

Cada noticia queda clasificada correctamente.

Caso de Prueba CP-04

Nombre: Georreferenciación

Objetivo: Validar la obtención de coordenadas geográficas.

Resultado esperado:

La noticia almacena latitud y longitud válidas.

Caso de Prueba CP-05

Nombre: Generación del archivo GeoJSON

Objetivo: Verificar la creación del archivo geográfico.

Resultado esperado:

Se genera un archivo GeoJSON con la estructura correcta y los eventos procesados.

Caso de Prueba CP-06

Nombre: Consulta en el mapa

Objetivo: Validar la visualización de los eventos georreferenciados.

Resultado esperado:

Los eventos aparecen correctamente representados sobre el mapa interactivo.

Caso de Prueba CP-07

Nombre: Consulta de estadísticas

Objetivo: Verificar la generación de indicadores estadísticos.

Resultado esperado:

Los gráficos presentan información consistente con los datos almacenados.

Caso de Prueba CP-08

Nombre: Generación de reportes

Objetivo: Validar la exportación de información.

Resultado esperado:

El sistema genera reportes en PDF, CSV o GeoJSON sin errores.

24.7 Matriz de Casos de Prueba

Código	Funcionalidad	Tipo de prueba	Resultado esperado
CP-01	Inicio de sesión	Funcional	Acceso correcto
CP-02	Captura RSS	Integración	Noticias almacenadas
CP-03	Clasificación	Funcional	Categorías correctas

Código	Funcionalidad	Tipo de prueba	Resultado esperado
CP-04	Georreferenciación	Funcional	Coordenadas válidas
CP-05	GeoJSON	Sistema	Archivo generado
CP-06	Mapa	Sistema	Eventos visibles
CP-07	Estadísticas	Funcional	Indicadores correctos
CP-08	Reportes	Funcional	Reportes exportados

24.8 Criterios de Aceptación

Una funcionalidad será considerada aprobada cuando:

- Cumpla el resultado esperado.
 - No presente errores críticos.
 - No afecte otros módulos del sistema.
 - Cumpla los tiempos de respuesta definidos.
 - Supere las pruebas de integración y aceptación.
-

24.9 Registro de Incidentes

Cada error identificado durante las pruebas será registrado con la siguiente información:

- Código del incidente.
 - Fecha.
 - Módulo afectado.
 - Descripción.
 - Nivel de severidad.
 - Responsable de la corrección.
 - Estado (Abierto, En proceso, Cerrado).
-

24.10 Herramientas de Prueba

Durante la ejecución de las pruebas podrán utilizarse las siguientes herramientas:

- **Pytest** para pruebas unitarias.
- **Postman** para validar la API REST.
- **MySQL Workbench** para verificar la base de datos.
- **GitHub** para el seguimiento de incidencias y control de versiones.

- **Visual Studio Code** para depuración del código.

24.11 Indicadores de Calidad

Se propone realizar seguimiento a los siguientes indicadores:

Indicador	Meta
Casos de prueba ejecutados	100 %
Casos de prueba aprobados	≥ 95 %
Cobertura de requisitos funcionales	100 %
Defectos críticos pendientes	0
Tiempo promedio de respuesta	≤ 3 segundos
Disponibilidad del sistema	≥ 99 %

24.12 Cronograma de Pruebas

Actividad	Semanas
Diseño de casos de prueba	13 - 14
Pruebas unitarias	15 - 16
Pruebas de integración	16 - 17
Pruebas funcionales	17 - 18
Pruebas de sistema	18 - 19
Pruebas de aceptación	20
Corrección de incidencias	20 - 21
Validación final	22

24.13 Roles y Responsabilidades

Rol	Responsabilidad
Director del Proyecto	Aprobar el plan de pruebas

Rol	Responsabilidad
Desarrolladores	Corregir errores detectados
Ingeniero QA	Diseñar y ejecutar las pruebas
Administrador del Sistema	Preparar el entorno de pruebas
Usuario Final	Realizar las pruebas de aceptación

24.14 Gestión de Riesgos en las Pruebas

Durante la fase de pruebas se controlarán riesgos como:

- Datos de prueba insuficientes.
- Cambios en los requisitos.
- Fallas en la infraestructura.
- Errores de integración.
- Baja cobertura de pruebas.
- Retrasos en la corrección de defectos.

Para mitigar estos riesgos se realizarán revisiones periódicas y seguimiento continuo de las incidencias.

24.15 Conclusión

El Plan de Pruebas de **Alert!** establece una estrategia integral para validar la calidad y el correcto funcionamiento de la plataforma. La ejecución de pruebas unitarias, de integración, funcionales, de sistema y de aceptación permitirá verificar el cumplimiento de los requisitos del proyecto, reducir el riesgo de fallos en producción y garantizar una solución confiable para el monitoreo automatizado de noticias, la generación de archivos GeoJSON y la visualización geoespacial de eventos.

Para complementar este capítulo en un proyecto de grado o del **SENA**, es recomendable anexar los siguientes documentos:

1. **Matriz de Trazabilidad de Requisitos**, relacionando cada requisito funcional con uno o más casos de prueba.
2. **Evidencias de Pruebas**, incluyendo capturas de pantalla y resultados obtenidos.
3. **Registro de Defectos**, con el historial de incidencias encontradas y su estado.
4. **Informe Final de Pruebas**, que resuma los resultados, las métricas de calidad y la aprobación para el despliegue del sistema.

Estos anexos fortalecerán la documentación técnica y demostrarán el proceso de aseguramiento de la calidad del software.

Con gusto. A continuación se presenta el capítulo **Plan de Despliegue** para el proyecto **Alert!**. Este documento está alineado con la arquitectura propuesta para la plataforma, la cual incluye una aplicación web desarrollada en Python/Flask, un proceso Batch para el monitoreo de noticias, una base de datos MySQL y la generación de archivos GeoJSON para su visualización en mapas.

25. Plan de Despliegue

25.1 Introducción

El Plan de Despliegue establece las actividades, procedimientos y recursos necesarios para instalar, configurar y poner en funcionamiento la plataforma **Alert!** en un ambiente de producción. Su objetivo es garantizar que la implementación del sistema se realice de forma controlada, segura y con el menor impacto posible para los usuarios finales.

El despliegue comprende la preparación de la infraestructura tecnológica, la instalación de los componentes de software, la configuración de la base de datos, la ejecución del proceso Batch, la validación del funcionamiento y el monitoreo posterior a la puesta en marcha.

25.2 Objetivos del Plan de Despliegue

Los objetivos del despliegue son:

- Instalar correctamente todos los componentes del sistema.
- Configurar el entorno de producción.
- Garantizar la disponibilidad de la aplicación.
- Verificar el funcionamiento de todos los módulos.
- Minimizar riesgos durante la puesta en marcha.
- Facilitar futuras actualizaciones del software.

25.3 Alcance

El plan contempla el despliegue de los siguientes componentes:

- Aplicación web Alert!.
- Servidor Flask.
- Base de datos MySQL.
- API REST.
- Proceso Batch.
- Archivo GeoJSON.
- Dashboard.
- Mapa interactivo.

- Sistema de autenticación.
 - Servicios de respaldo.
-

25.4 Infraestructura Requerida

Servidor de Aplicaciones

Características recomendadas:

- Sistema Operativo: Ubuntu Server 24.04 LTS o Windows Server.
 - Procesador: mínimo 4 núcleos.
 - Memoria RAM: 8 GB o superior.
 - Almacenamiento SSD: 100 GB o superior.
 - Conectividad permanente a Internet.
-

Base de Datos

- MySQL 8.x
 - Copias de seguridad automáticas.
 - Acceso restringido.
 - Configuración de usuarios y permisos.
-

Servidor Web

Se recomienda utilizar:

- Gunicorn (Linux).
 - Nginx como servidor proxy inverso.
 - HTTPS mediante certificado SSL.
 - Firewall configurado.
-

25.5 Requisitos Previos

Antes del despliegue se deberá verificar:

- Python instalado.
- Entorno virtual configurado.
- Dependencias instaladas desde requirements.txt.
- Base de datos creada.
- Variables de entorno configuradas.

- Acceso al servidor.
 - Dominio registrado (si aplica).
 - Certificado SSL instalado.
-

25.6 Procedimiento de Despliegue

Paso 1. Preparación del Servidor

- Actualizar el sistema operativo.
 - Instalar Python.
 - Instalar MySQL.
 - Instalar Git.
 - Instalar Gunicorn.
 - Configurar Nginx.
 - Crear el entorno virtual.
-

Paso 2. Instalación del Proyecto

- Clonar el repositorio desde GitHub.
 - Crear el entorno virtual.
 - Instalar las dependencias.
 - Configurar las variables de entorno.
 - Ajustar los permisos de archivos y carpetas.
-

Paso 3. Configuración de la Base de Datos

- Crear la base de datos.
 - Ejecutar los scripts SQL.
 - Crear usuarios con permisos adecuados.
 - Cargar datos iniciales.
 - Verificar la conectividad desde la aplicación.
-

Paso 4. Configuración del Proceso Batch

El proceso Batch deberá programarse para ejecutarse automáticamente mediante:

Linux

- Cron.

Windows

- Programador de tareas (Task Scheduler).

Las tareas programadas deberán:

- Consultar fuentes RSS.
 - Procesar noticias.
 - Clasificar eventos.
 - Georreferenciar información.
 - Generar el archivo GeoJSON.
 - Actualizar la base de datos.
-

Paso 5. Configuración del Portal Web

Se verificará:

- Inicio de sesión.
 - Dashboard.
 - Consulta de noticias.
 - Mapa interactivo.
 - Reportes.
 - Estadísticas.
 - API REST.
-

Paso 6. Validación

Se ejecutarán pruebas para comprobar:

- Acceso al sistema.
 - Funcionamiento del proceso Batch.
 - Generación correcta del GeoJSON.
 - Integridad de la base de datos.
 - Rendimiento del servidor.
 - Seguridad de la aplicación.
-

25.7 Estrategia de Despliegue

El despliegue se realizará en tres ambientes independientes:

Ambiente de Desarrollo

Utilizado por los desarrolladores para implementar nuevas funcionalidades y realizar pruebas iniciales.

Ambiente de Pruebas

Destinado a validar la integración, el rendimiento y el funcionamiento del sistema antes de su publicación.

Ambiente de Producción

Ambiente utilizado por los usuarios finales para la operación oficial de la plataforma.

25.8 Plan de Migración

En caso de actualizar una versión existente, el procedimiento será:

1. Realizar copia de seguridad.
 2. Suspendar temporalmente el servicio.
 3. Actualizar el código fuente.
 4. Ejecutar migraciones de la base de datos.
 5. Reiniciar los servicios.
 6. Ejecutar pruebas de validación.
 7. Habilitar nuevamente el acceso a los usuarios.
-

25.9 Respaldo y Recuperación

El plan contempla:

- Copias de seguridad diarias de la base de datos.
 - Respaldo de archivos GeoJSON.
 - Respaldo del código fuente.
 - Copias de archivos de configuración.
 - Procedimientos de restauración documentados.
-

25.10 Monitoreo

Después del despliegue se realizará monitoreo permanente de:

- Disponibilidad del servidor.
- Uso de CPU y memoria.
- Espacio en disco.
- Estado del proceso Batch.
- Errores de la aplicación.
- Rendimiento de la base de datos.
- Estado de la API REST.

25.11 Plan de Contingencia

En caso de presentarse fallos durante el despliegue se ejecutarán las siguientes acciones:

- Restaurar la copia de seguridad.
- Revertir a la versión anterior del software.
- Revisar los registros (logs) del sistema.
- Corregir la configuración afectada.
- Repetir las pruebas de validación.
- Notificar a los usuarios sobre el estado del servicio.

25.12 Roles y Responsabilidades

Rol	Responsabilidades
Director del Proyecto	Aprobar el despliegue
Administrador del Sistema	Configurar servidores e infraestructura
Desarrolladores	Instalar y configurar la aplicación
Administrador de Base de Datos	Implementar y validar la base de datos
Ingeniero QA	Ejecutar pruebas posteriores al despliegue
Usuario Final	Validar el funcionamiento del sistema

25.13 Cronograma de Despliegue

Actividad	Duración
-----------	----------

Actividad	Duración
Preparación del servidor	1 día
Instalación de dependencias	1 día
Configuración de la base de datos	1 día
Instalación de la aplicación	1 día
Configuración del proceso Batch	1 día
Pruebas de validación	2 días
Puesta en producción	1 día
Monitoreo inicial	5 días

Duración estimada del despliegue: **13 días calendario.**

25.14 Criterios de Aceptación

El despliegue será considerado exitoso cuando:

- La aplicación esté disponible para los usuarios.
- El proceso Batch se ejecute correctamente.
- Se generen archivos GeoJSON válidos.
- El mapa interactivo muestre la información procesada.
- La base de datos funcione sin errores.
- Se cumplan los tiempos de respuesta definidos.
- No existan defectos críticos pendientes.

25.15 Indicadores de Éxito

Indicador	Meta
Disponibilidad del sistema	$\geq 99 \%$
Tiempo de respuesta	≤ 3 segundos
Ejecuciones exitosas del Batch	$\geq 95 \%$
Generación correcta de GeoJSON	100 %
Incidentes críticos posteriores al despliegue	0

Indicador	Meta
Recuperación ante fallos	≤ 30 minutos

25.16 Conclusión

El Plan de Despliegue de la plataforma **Alert!** define una estrategia estructurada para la instalación, configuración y puesta en operación del sistema. La implementación de ambientes separados, procedimientos de validación, mecanismos de respaldo y monitoreo continuo permitirá reducir los riesgos asociados a la publicación del software y garantizar un servicio estable para el monitoreo automatizado de noticias, la georreferenciación de eventos y la generación de archivos GeoJSON.

Como complemento a este capítulo, es recomendable anexar los siguientes documentos técnicos:

1. **Manual de Instalación**, con los pasos detallados para desplegar la aplicación en Linux y Windows.
2. **Guía de Configuración del Servidor**, incluyendo Python, Flask, Gunicorn, Nginx y MySQL.
3. **Scripts de Automatización** (por ejemplo, Bash o Batch) para facilitar la instalación y las actualizaciones.
4. **Diagrama de Despliegue UML**, mostrando la distribución de componentes entre el cliente, el servidor de aplicaciones, la base de datos y el proceso Batch.

Estos anexos fortalecerán la documentación técnica y facilitarán futuras instalaciones, actualizaciones y tareas de mantenimiento del sistema.

26. Plan de Mantenimiento

26.1 Introducción

El Plan de Mantenimiento establece las actividades necesarias para asegurar el correcto funcionamiento de la plataforma **Alert!** durante su ciclo de vida. El mantenimiento tiene como propósito corregir errores, optimizar el rendimiento, adaptar el sistema a nuevos requerimientos tecnológicos y garantizar la disponibilidad del servicio.

Debido a que **Alert!** monitorea noticias de fuentes RSS, ejecuta procesos Batch, clasifica información, genera archivos GeoJSON y ofrece visualización geoespacial mediante un portal web, es indispensable contar con un plan que permita mantener la plataforma actualizada, segura y eficiente.

26.2 Objetivos del Plan de Mantenimiento

Los objetivos del mantenimiento son:

- Garantizar la disponibilidad y estabilidad del sistema.
 - Corregir errores detectados durante la operación.
 - Mejorar continuamente el rendimiento de la aplicación.
 - Adaptar el sistema a nuevas tecnologías y cambios en las fuentes de información.
 - Mantener la seguridad de la plataforma.
 - Asegurar la integridad de la información almacenada.
-

26.3 Alcance

El plan de mantenimiento comprende los siguientes componentes:

- Aplicación web Alert!
 - API REST
 - Base de datos MySQL
 - Proceso Batch
 - Generación de archivos GeoJSON
 - Dashboard
 - Mapa interactivo
 - Sistema de autenticación
 - Infraestructura tecnológica
 - Documentación técnica
-

26.4 Tipos de Mantenimiento

Mantenimiento Correctivo

Tiene como finalidad corregir errores identificados durante la operación del sistema.

Ejemplos:

- Corrección de errores de programación.
 - Solución de fallos en el proceso Batch.
 - Corrección de consultas SQL.
 - Reparación de errores en la API REST.
 - Ajustes en la generación de archivos GeoJSON.
-

Mantenimiento Preventivo

Busca reducir la probabilidad de fallos mediante actividades programadas.

Actividades:

- Actualización de dependencias.
 - Optimización de la base de datos.
 - Limpieza de archivos temporales.
 - Revisión de registros (logs).
 - Verificación del funcionamiento del servidor.
 - Validación de copias de seguridad.
-

Mantenimiento Adaptativo

Permite ajustar el sistema ante cambios en el entorno tecnológico.

Ejemplos:

- Actualización de versiones de Python o Flask.
 - Cambios en las fuentes RSS.
 - Incorporación de nuevas APIs de geolocalización.
 - Compatibilidad con nuevas versiones de navegadores.
-

Mantenimiento Perfectivo

Consiste en mejorar el desempeño y la experiencia del usuario.

Ejemplos:

- Optimización del tiempo de respuesta.
 - Mejoras en la interfaz gráfica.
 - Nuevos filtros de búsqueda.
 - Nuevos reportes y estadísticas.
 - Incorporación de nuevas funcionalidades.
-

26.5 Actividades de Mantenimiento

Las actividades principales incluyen:

- Monitorear el estado del servidor.
- Revisar la ejecución del proceso Batch.

- Verificar la disponibilidad de las fuentes RSS.
 - Validar la correcta generación del archivo GeoJSON.
 - Optimizar consultas SQL.
 - Actualizar bibliotecas y dependencias.
 - Revisar registros de errores.
 - Ejecutar copias de seguridad.
 - Restaurar información cuando sea necesario.
 - Actualizar la documentación técnica.
-

26.6 Frecuencia de las Actividades

Actividad	Frecuencia
Monitoreo del servidor	Diario
Ejecución del proceso Batch	Según programación (por ejemplo, cada hora o diariamente)
Revisión de registros (logs)	Diario
Copias de seguridad de la base de datos	Diario
Verificación de la generación del archivo GeoJSON	Diario
Optimización de la base de datos	Mensual
Actualización de dependencias	Trimestral
Revisión de seguridad	Trimestral
Auditoría técnica	Semestral

26.7 Procedimiento de Mantenimiento

El proceso de mantenimiento comprende las siguientes etapas:

1. Recepción del incidente o solicitud de mejora.
2. Registro y clasificación.
3. Análisis de impacto.
4. Priorización.
5. Desarrollo de la solución.
6. Pruebas en ambiente de desarrollo.
7. Validación en ambiente de pruebas.
8. Implementación en producción.
9. Documentación de los cambios.

10. Cierre del mantenimiento.

26.8 Gestión de Incidentes

Todo incidente deberá registrarse con la siguiente información:

- Código del incidente.
 - Fecha de reporte.
 - Módulo afectado.
 - Descripción.
 - Nivel de prioridad.
 - Responsable asignado.
 - Estado (Abierto, En análisis, En desarrollo, Resuelto, Cerrado).
 - Fecha de cierre.
-

26.9 Priorización de Incidentes

Prioridad	Descripción	Tiempo objetivo de atención
Crítica	El sistema no funciona o existe pérdida de información	Menos de 4 horas
Alta	Funcionalidad principal afectada	1 día hábil
Media	Error con alternativa de operación	3 días hábiles
Baja	Mejoras o ajustes menores	Hasta 10 días hábiles

26.10 Copias de Seguridad

Se implementará una política de respaldo para garantizar la recuperación de la información.

Base de datos

- Respaldo automático diario.
- Conservación de respaldos durante 30 días.

Código fuente

- Almacenamiento en GitHub.

- Control de versiones mediante Git.
- Etiquetado de versiones estables.

Archivos GeoJSON

- Copia diaria.
 - Almacenamiento en servidor secundario o servicio en la nube.
-

26.11 Actualización del Software

Las actualizaciones deberán seguir el siguiente procedimiento:

1. Desarrollo de mejoras.
 2. Ejecución de pruebas unitarias.
 3. Pruebas de integración.
 4. Aprobación del cambio.
 5. Despliegue en producción.
 6. Monitoreo posterior a la actualización.
-

26.12 Monitoreo del Sistema

Se realizará seguimiento permanente a:

- Disponibilidad del servidor.
 - Uso de CPU y memoria.
 - Espacio en disco.
 - Estado de la base de datos.
 - Estado del proceso Batch.
 - Errores de la aplicación.
 - Tiempo de respuesta de la API REST.
 - Generación de archivos GeoJSON.
-

26.13 Roles y Responsabilidades

Rol	Responsabilidades
Administrador del Sistema	Monitorear servidores, respaldos y disponibilidad
Desarrollador Backend	Corregir errores del servidor, API y proceso Batch
Desarrollador Frontend	Mantener la interfaz web y el Dashboard

Rol	Responsabilidades
Administrador de Base de Datos	Optimizar y respaldar la base de datos
Ingeniero QA	Validar las correcciones antes de su publicación
Director del Proyecto	Aprobar cambios mayores y supervisar el mantenimiento

26.14 Indicadores de Mantenimiento

Indicador	Meta
Disponibilidad del sistema	$\geq 99 \%$
Tiempo promedio de respuesta	≤ 3 segundos
Tiempo de recuperación ante fallos críticos	≤ 4 horas
Copias de seguridad exitosas	100 %
Incidentes críticos resueltos dentro del tiempo objetivo	$\geq 95 \%$
Actualizaciones implementadas sin errores	$\geq 95 \%$

26.15 Herramientas de Apoyo

Para ejecutar el mantenimiento del sistema se podrán utilizar las siguientes herramientas:

- Git y GitHub para el control de versiones.
 - Visual Studio Code para el desarrollo.
 - MySQL Workbench para la administración de la base de datos.
 - Postman para validar la API REST.
 - Pytest para pruebas automatizadas.
 - Herramientas de monitoreo del servidor y análisis de registros (logs).
-

26.16 Plan de Mejora Continua

Con el fin de mantener la competitividad y la calidad de la plataforma, se establecerán actividades periódicas de mejora continua:

- Revisión de sugerencias de los usuarios.

- Incorporación de nuevas fuentes RSS.
 - Optimización de los algoritmos de clasificación.
 - Mejora de la precisión en la georreferenciación.
 - Actualización de la interfaz de usuario.
 - Optimización del rendimiento de consultas y reportes.
 - Evaluación de nuevas tecnologías y bibliotecas.
-

26.17 Conclusión

El Plan de Mantenimiento de la plataforma **Alert!** proporciona una estrategia integral para asegurar la continuidad operativa, la disponibilidad y la evolución del sistema. La combinación de mantenimiento correctivo, preventivo, adaptativo y perfectivo permitirá responder de manera oportuna a incidentes, incorporar mejoras tecnológicas y mantener un servicio confiable para el monitoreo automatizado de noticias, la generación de archivos GeoJSON y la visualización geoespacial de eventos.

Con gusto. A continuación se presenta el capítulo **Resultados Esperados** para la formulación del proyecto de software **Alert!**. Este apartado describe los productos, beneficios e impactos que se esperan obtener una vez finalizado el desarrollo e implementación de la plataforma.

27. Resultados Esperados

27.1 Introducción

Los resultados esperados representan los productos, capacidades y beneficios que se espera alcanzar con el desarrollo e implementación de la plataforma **Alert!**. Estos resultados están alineados con los objetivos del proyecto y permiten evaluar el grado de cumplimiento de las metas propuestas.

La plataforma tiene como propósito automatizar el monitoreo de noticias provenientes de diversas fuentes RSS, clasificar los eventos mediante criterios previamente definidos, georreferenciar la información y generar archivos GeoJSON que faciliten su visualización y análisis en mapas interactivos.

27.2 Resultado General

Se espera desarrollar una plataforma web funcional denominada **Alert!**, capaz de monitorear de forma automática noticias publicadas en Internet, procesarlas mediante un sistema Batch, clasificarlas según su nivel de severidad, georreferenciarlas y presentar la información mediante mapas interactivos y paneles estadísticos, apoyando la consulta y el análisis de eventos relevantes.

27.3 Resultados Específicos

Al finalizar el proyecto se espera obtener los siguientes resultados:

1. Plataforma Web Funcional

Implementación de una aplicación web que permita:

- Consultar noticias procesadas.
- Visualizar mapas interactivos.
- Generar reportes.
- Consultar estadísticas.
- Administrar usuarios y fuentes RSS.

2. Automatización del Monitoreo

Desarrollo de un proceso Batch que permita:

- Consultar múltiples fuentes RSS.
- Descargar automáticamente nuevas noticias.
- Evitar registros duplicados.
- Procesar grandes volúmenes de información de manera periódica.

3. Clasificación Automática de Noticias

Implementación de mecanismos para:

- Clasificar noticias por categorías.
- Asignar niveles de severidad.
- Organizar la información para facilitar su análisis.

4. Georreferenciación de Eventos

Obtención de coordenadas geográficas asociadas a los lugares mencionados en las noticias, permitiendo su representación espacial.

5. Generación de Archivos GeoJSON

Producción automática de archivos GeoJSON compatibles con herramientas de cartografía digital, facilitando su integración con aplicaciones SIG y bibliotecas como Leaflet.

6. Base de Datos Estructurada

Implementación de una base de datos relacional para almacenar:

- Noticias.
 - Categorías.
 - Usuarios.
 - Fuentes RSS.
 - Coordenadas geográficas.
 - Registros históricos.
 - Resultados del procesamiento Batch.
-

7. Dashboard Interactivo

Desarrollo de un panel de control con indicadores que permitan visualizar:

- Noticias procesadas.
 - Eventos por categoría.
 - Distribución geográfica.
 - Tendencias temporales.
 - Nivel de severidad de los eventos.
-

8. API REST

Implementación de una API que facilite el acceso controlado a la información procesada por parte de otros sistemas o aplicaciones.

9. Reportes Automatizados

Generación de reportes exportables en formatos como:

- PDF.
 - CSV.
 - GeoJSON.
-

10. Documentación Técnica

Elaboración de la documentación del proyecto, incluyendo:

- Formulación del proyecto.
 - Arquitectura del sistema.
 - Diseño de base de datos.
 - Diagramas UML.
 - Manual técnico.
 - Manual de usuario.
 - Planes de pruebas, despliegue y mantenimiento.
-

27.4 Beneficios Esperados

La implementación de la plataforma permitirá:

- Automatizar el monitoreo de información publicada en medios digitales.
 - Reducir el tiempo requerido para identificar eventos relevantes.
 - Centralizar información proveniente de múltiples fuentes.
 - Mejorar la organización y consulta de datos.
 - Facilitar el análisis geográfico mediante mapas interactivos.
 - Disponer de información histórica para apoyar estudios y análisis posteriores.
 - Favorecer la toma de decisiones basada en datos.
-

27.5 Indicadores de Cumplimiento

Indicador	Meta Esperada
Módulos desarrollados	100 %
Requisitos funcionales implementados	100 %
Casos de prueba aprobados	≥ 95 %
Ejecuciones exitosas del proceso Batch	≥ 95 %
Generación correcta de archivos GeoJSON	100 %
Disponibilidad de la plataforma	≥ 99 %
Tiempo promedio de respuesta	≤ 3 segundos
Reportes generados correctamente	100 %

27.6 Impacto Tecnológico

Se espera que el proyecto contribuya al fortalecimiento de competencias en:

- Desarrollo de aplicaciones web con Python y Flask.
 - Integración de servicios mediante API REST.
 - Procesamiento automatizado de información.
 - Manejo de bases de datos relacionales.
 - Georreferenciación y análisis espacial.
 - Visualización de datos geográficos.
 - Implementación de procesos Batch.
 - Aplicación de metodologías ágiles de desarrollo.
-

27.7 Impacto Académico

Como proyecto de formación, **Alert!** permitirá:

- Aplicar conocimientos adquiridos durante el proceso académico.
 - Integrar diferentes áreas de la ingeniería de software.
 - Generar documentación técnica completa.
 - Fortalecer competencias en análisis, diseño, desarrollo y pruebas de software.
 - Servir como referencia para futuros proyectos relacionados con monitoreo de información y análisis geoespacial.
-

27.8 Impacto Organizacional y Social

La plataforma podrá aportar beneficios a organizaciones públicas y privadas que requieran monitorear información publicada en medios digitales, facilitando el seguimiento de eventos y la consulta de información georreferenciada.

Asimismo, el sistema podrá apoyar actividades de análisis, investigación y gestión de información mediante la consolidación de datos provenientes de diversas fuentes y su representación visual.

27.9 Criterios de Éxito

El proyecto se considerará exitoso cuando:

- Todos los módulos definidos en el alcance se encuentren implementados.
- El proceso Batch funcione de manera automática y estable.

- La clasificación y georreferenciación de noticias se ejecuten correctamente.
 - Los archivos GeoJSON sean generados con una estructura válida.
 - La plataforma permita consultar la información mediante mapas y reportes.
 - Se complete la documentación técnica y funcional del sistema.
 - Las pruebas de aceptación sean aprobadas por los usuarios definidos para el proyecto.
-

27.10 Conclusión

Los resultados esperados del proyecto **Alert!** reflejan el cumplimiento de los objetivos planteados durante su formulación y establecen los productos que se obtendrán como resultado del desarrollo. La plataforma proporcionará una solución tecnológica para el monitoreo automatizado de noticias, el procesamiento Batch, la georreferenciación de eventos y la generación de información geoespacial, apoyando la consulta, el análisis y la gestión de información mediante una aplicación web moderna, escalable y documentada.

Como complemento a este capítulo, se recomienda incluir una **matriz de trazabilidad**, donde cada objetivo específico se relacione con los entregables, los indicadores de cumplimiento y los resultados esperados. Esto facilitará demostrar que el proyecto cumple con los objetivos definidos en la formulación y fortalecerá la evaluación final del trabajo.

28. Impacto Social, Tecnológico y Económico

28.1 Introducción

El desarrollo de la plataforma **Alert!** representa una iniciativa tecnológica orientada a mejorar el monitoreo automatizado de noticias mediante la recopilación de información desde múltiples fuentes RSS, el procesamiento Batch, la clasificación de eventos, la georreferenciación y la generación de archivos GeoJSON.

La implementación de esta solución tiene el potencial de generar beneficios en diferentes ámbitos, al facilitar el acceso a información organizada, mejorar los procesos de análisis de datos y promover el uso de tecnologías de código abierto para la gestión de información geoespacial.

28.2 Impacto Social

La plataforma **Alert!** contribuirá al fortalecimiento del acceso y análisis de la información, permitiendo a organizaciones, investigadores y usuarios consultar de manera centralizada noticias provenientes de diferentes fuentes digitales.

Entre los principales impactos sociales se destacan:

- Acceso oportuno a información organizada sobre eventos relevantes.
- Consolidación de noticias provenientes de múltiples medios de información.
- Apoyo al análisis de tendencias mediante la representación geográfica de los eventos.
- Fortalecimiento de la cultura de uso de datos para la toma de decisiones.
- Disponibilidad de información histórica para actividades académicas e investigativas.
- Apoyo a entidades públicas y privadas que requieren monitorear información publicada en medios digitales.
- Promoción del uso de tecnologías geoespaciales para el análisis de información.

Asimismo, la plataforma puede convertirse en una herramienta de apoyo para proyectos de investigación, observatorios, centros de análisis y organizaciones interesadas en el seguimiento de acontecimientos de interés público.

28.3 Impacto Tecnológico

Desde el punto de vista tecnológico, **Alert!** integra diferentes herramientas y metodologías de desarrollo de software, contribuyendo a la construcción de una solución moderna, escalable y mantenible.

Los principales aportes tecnológicos incluyen:

- Automatización del monitoreo de noticias mediante procesos Batch.
- Integración de múltiples fuentes RSS en una única plataforma.
- Implementación de una arquitectura basada en Python y Flask.
- Desarrollo de una API REST para facilitar la interoperabilidad con otros sistemas.
- Utilización de bases de datos relacionales para el almacenamiento estructurado de la información.
- Generación automática de archivos GeoJSON para su integración con herramientas de cartografía digital.
- Visualización interactiva de datos mediante mapas y paneles estadísticos.
- Uso de herramientas de código abierto que facilitan la sostenibilidad y evolución del sistema.
- Diseño modular que permite incorporar nuevas funcionalidades y fuentes de información en el futuro.

Estas características favorecen la reutilización del software y facilitan su adaptación a diferentes contextos de uso.

28.4 Impacto Económico

La implementación de la plataforma puede generar beneficios económicos al optimizar procesos relacionados con la recopilación, organización y consulta de información.

Entre los principales impactos económicos se encuentran:

- Reducción del tiempo requerido para recopilar noticias desde múltiples fuentes.
- Disminución de tareas manuales mediante la automatización del procesamiento de información.
- Aprovechamiento de tecnologías de código abierto, reduciendo costos de licenciamiento.
- Optimización de recursos tecnológicos mediante procesos automáticos de actualización y clasificación.
- Disminución de costos asociados a la búsqueda y consolidación manual de información.
- Posibilidad de reutilizar la plataforma en nuevos proyectos, reduciendo tiempos y costos de desarrollo.

En un entorno organizacional, estas mejoras pueden traducirse en procesos más eficientes para el seguimiento y análisis de información.

28.5 Impacto Académico

El proyecto también genera un impacto positivo en el ámbito académico al integrar conocimientos de diferentes áreas de la ingeniería de software.

Entre sus aportes se destacan:

- Aplicación práctica de metodologías para el desarrollo de software.
 - Integración de bases de datos, servicios web, procesamiento Batch y georreferenciación.
 - Elaboración de documentación técnica completa.
 - Desarrollo de competencias en análisis, diseño, implementación, pruebas y mantenimiento de software.
 - Posibilidad de utilizar la plataforma como caso de estudio o base para investigaciones futuras.
-

28.6 Impacto Ambiental

El proyecto presenta un impacto ambiental reducido debido a su naturaleza digital.

Entre los aspectos positivos se encuentran:

- Disminución del uso de documentos impresos mediante la generación de reportes digitales.
 - Promoción del almacenamiento electrónico de información.
 - Optimización del uso de recursos tecnológicos mediante procesos automatizados.
 - Posibilidad de implementar infraestructura virtualizada o servicios en la nube para mejorar la eficiencia en el uso de recursos informáticos.
-

28.7 Beneficiarios del Proyecto

Los principales beneficiarios de la plataforma **Alert!** son:

Beneficiarios directos

- Analistas de información.
- Investigadores.
- Administradores del sistema.
- Organizaciones públicas y privadas que requieran monitorear noticias.

Beneficiarios indirectos

- Instituciones educativas.
 - Centros de investigación.
 - Entidades gubernamentales.
 - Empresas interesadas en el análisis de información.
 - Comunidad académica.
-

28.8 Sostenibilidad del Proyecto

La sostenibilidad de la plataforma estará respaldada por:

- Uso de software libre y de código abierto.
 - Arquitectura modular y escalable.
 - Documentación técnica y funcional.
 - Control de versiones mediante Git.
 - Planes de mantenimiento y actualización.
 - Posibilidad de incorporar nuevas funcionalidades sin afectar la estructura principal del sistema.
-

28.9 Conclusión

La plataforma **Alert!** presenta un impacto positivo en los ámbitos social, tecnológico y económico al ofrecer una solución para el monitoreo automatizado de noticias y la organización de información georreferenciada. La integración de procesos Batch, tecnologías de código abierto, bases de datos relacionales, API REST y archivos GeoJSON favorece la eficiencia en el tratamiento de la información, la interoperabilidad con otros sistemas y la evolución futura de la plataforma. Además, el proyecto aporta valor académico y organizacional al servir como referencia para el desarrollo de soluciones orientadas al análisis y visualización de información geoespacial.

29. Conclusiones

29.1 Conclusiones

El desarrollo de la formulación del proyecto **Alert!** permitió estructurar de manera integral una propuesta de software orientada al monitoreo automatizado de noticias, mediante la integración de procesos Batch, georreferenciación y visualización de información geográfica. La documentación elaborada constituye una guía técnica y metodológica para el diseño, desarrollo, implementación y mantenimiento de la plataforma.

A partir del análisis del problema se identificó la necesidad de contar con una herramienta que automatice la recopilación de información proveniente de múltiples fuentes RSS, reduzca los tiempos de procesamiento y facilite el análisis de eventos mediante información organizada y georreferenciada. La propuesta responde a esta necesidad mediante una arquitectura basada en tecnologías de código abierto, que favorecen la escalabilidad, el mantenimiento y la sostenibilidad del sistema.

Durante la formulación se definieron los objetivos, el alcance, los requisitos funcionales y no funcionales, la arquitectura del sistema, el modelo de datos, los diagramas UML, el diseño de interfaces y los procesos de desarrollo, proporcionando una base sólida para la construcción del software. Asimismo, la adopción de una metodología ágil como Scrum favorece la planificación iterativa, el seguimiento del avance y la incorporación controlada de mejoras.

Uno de los principales aportes del proyecto es la automatización del procesamiento de noticias mediante procesos Batch, capaces de consultar periódicamente diversas fuentes RSS, clasificar la información, identificar ubicaciones geográficas y generar archivos GeoJSON compatibles con herramientas de cartografía digital. Esta funcionalidad permite representar espacialmente los eventos y facilita el análisis mediante mapas interactivos y paneles estadísticos.

La utilización de una arquitectura modular basada en Python, Flask, MySQL y una API REST proporciona una solución flexible y escalable, preparada para incorporar nuevas funcionalidades, ampliar el número de fuentes de información e integrarse con otros sistemas. El empleo de tecnologías de código abierto contribuye además a disminuir los costos de implementación y facilita la continuidad del proyecto.

Los capítulos dedicados a la gestión de riesgos, el plan de pruebas, el plan de despliegue y el plan de mantenimiento evidencian la importancia de contemplar todo el ciclo de vida del software. Estas actividades permiten fortalecer la calidad del producto, reducir riesgos técnicos y operativos, asegurar la estabilidad de la plataforma y facilitar su evolución futura mediante procesos documentados y controlados.

Desde la perspectiva académica, el proyecto integra conocimientos relacionados con ingeniería de software, análisis y diseño de sistemas, bases de datos, desarrollo web, procesamiento automatizado de información, servicios web, georreferenciación y gestión de proyectos. La documentación generada constituye un referente para proyectos similares y demuestra la aplicación práctica de metodologías y buenas prácticas de desarrollo.

En el ámbito tecnológico, **Alert!** propone una solución innovadora que combina el monitoreo automatizado de información, el procesamiento de datos, la representación geoespacial y la visualización interactiva de resultados. Esta integración favorece el análisis de información proveniente de múltiples fuentes y proporciona una plataforma adaptable a diferentes escenarios organizacionales y académicos.

Finalmente, se concluye que la plataforma **Alert!** es técnica, funcional y metodológicamente viable. Su implementación permitirá disponer de una herramienta capaz de automatizar el monitoreo de noticias, organizar la información de manera estructurada, representar los eventos mediante mapas interactivos y generar información útil para el análisis y la toma de decisiones. La formulación desarrollada establece las bases necesarias para continuar con la implementación, validación y evolución del sistema, consolidando un proyecto con potencial de crecimiento y aplicación en diferentes contextos donde el análisis oportuno de información georreferenciada sea un elemento de apoyo para la gestión del conocimiento.

15. Recomendaciones

Con el propósito de garantizar la sostenibilidad, escalabilidad y evolución del proyecto **Alert!**, se recomienda tener en cuenta las siguientes consideraciones durante su desarrollo e implementación:

1. **Ampliar las fuentes de información.** Incorporar progresivamente nuevos canales RSS, APIs de noticias y fuentes oficiales nacionales e internacionales para mejorar la cobertura y diversidad de la información procesada.
2. **Optimizar el procesamiento Batch.** Configurar la ejecución periódica de los procesos por lotes mediante un planificador de tareas (Scheduler) que permita actualizar automáticamente la información sin intervención manual.
3. **Implementar técnicas de Inteligencia Artificial.** Integrar modelos de Procesamiento de Lenguaje Natural (NLP) y aprendizaje automático para mejorar la clasificación automática de noticias, la identificación de entidades geográficas y la asignación de niveles de severidad.
4. **Implementar una base de datos geoespacial.** Utilizar gestores especializados como PostgreSQL con la extensión PostGIS para almacenar y consultar eficientemente la información contenida en los archivos GeoJSON generados por el sistema.

5. **Fortalecer la seguridad de la plataforma.** Implementar mecanismos de autenticación, autorización, cifrado de información, copias de seguridad y auditoría para garantizar la integridad y disponibilidad de los datos.
6. **Desarrollar una API pública.** Facilitar la interoperabilidad con otros sistemas mediante servicios REST que permitan consultar alertas, incidentes históricos y estadísticas generadas por la plataforma.
7. **Implementar tableros de inteligencia.** Integrar herramientas de visualización que permitan analizar tendencias, concentraciones geográficas de incidentes y evolución temporal de los eventos registrados.
8. **Realizar pruebas periódicas.** Ejecutar pruebas funcionales, de rendimiento, seguridad y usabilidad para asegurar el correcto funcionamiento de la plataforma antes de cada nueva versión.
9. **Mantener actualizado el diccionario de categorías.** Revisar continuamente las palabras clave, categorías y criterios de clasificación con el fin de mejorar la precisión del sistema frente a nuevos tipos de incidentes.
10. **Documentar las nuevas funcionalidades.** Mantener actualizada la documentación técnica y de usuario para facilitar el mantenimiento, la evolución del sistema y la incorporación de nuevos desarrolladores.

La aplicación de estas recomendaciones contribuirá a consolidar Alert! como una plataforma robusta, escalable y confiable para el monitoreo automatizado, la georreferenciación y el análisis inteligente de noticias.

Bibliografía (Norma APA 7.^a edición)

Berners-Lee, T., Fielding, R., & Masinter, L. (2005). *Uniform Resource Identifier (URI): Generic syntax* (RFC 3986). Internet Engineering Task Force.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation). University of California, Irvine.

GeoJSON Working Group. (2016). *The GeoJSON format* (RFC 7946). Internet Engineering Task Force.

International Organization for Standardization. (2011). *ISO/IEC 25010:2011. Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models*. ISO.

Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería del software: Un enfoque práctico* (9.^a ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Ingeniería de software* (10.^a ed.). Pearson Educación.

Stallings, W. (2020). *Computer networking with Internet protocols and technology*. Pearson.

The Linux Foundation. (2024). *GeoJSON specification*. <https://geojson.org/>

Van Rossum, G., & Drake, F. L. (2009). *The Python language reference*. Python Software Foundation.

W3C. (2004). *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. World Wide Web Consortium.

Python Software Foundation. (2026). *Python documentation*. <https://docs.python.org/3/>

Flask Development Team. (2026). *Flask documentation*.
<https://flask.palletsprojects.com/>

Leaflet Contributors. (2026). *Leaflet documentation*. <https://leafletjs.com/>

OpenStreetMap Foundation. (2026). *OpenStreetMap documentation*.
<https://www.openstreetmap.org/>

Instituto Geográfico Agustín Codazzi. (2023). *Infraestructura Colombiana de Datos Espaciales (ICDE)*. <https://www.icde.gov.co/>

Camilo Torrecillas Ardila.-

Tecnología en análisis y desarrollo de software.
